Double Linked List and Circular Linked List

Introduction to Doubly Linked List

- We have discussed the details of linear linked list. In the linear linked list, we can only traverse the linked list in **one direction**.
- But sometimes, it is very desirable to traverse a linked list in either a **forward** or **reverse** manner.
- This property of a linked list implies that each node must contain **two link fields** instead of one.
- The links are used to denote the **predecessor** and **successor** of a node.
- The link denoting the predecessor of a node is called the **left link**, and that denoting its successor its **right link**

Basic Operation of Doubly Linked List

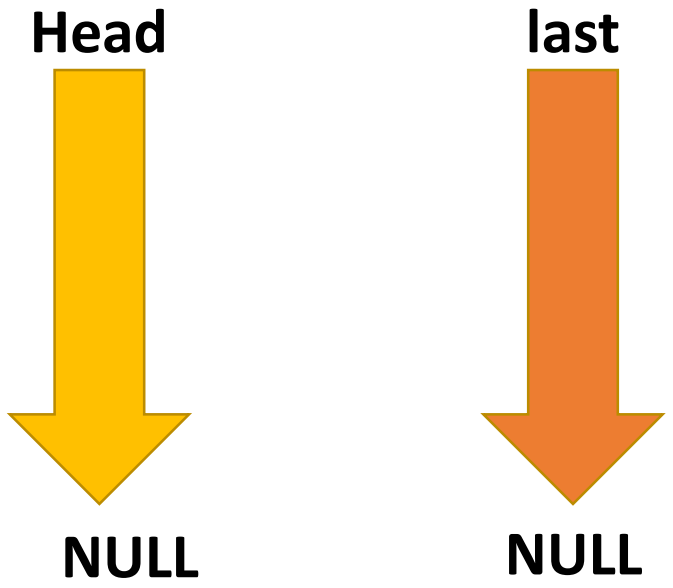
- Insert: Add a new node in the first, last or interior of the list.
- Delete: Delete a node from the first, last or interior of the list.
- Search: Search a node containing particular value in the linked list.

Insertion

- Insertion is to add a new node into a linked list. It can take place anywhere:
  - The first position
  - The last position
  - Any interior position

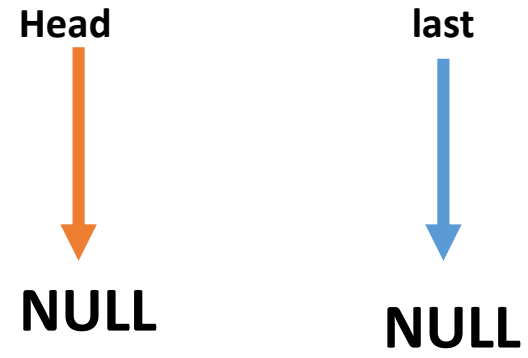
Initial Scenatio

```
struct dnode{
    int value;
    struct dnode *prev;
    struct dnode *next;
};

struct dnode *head = NULL;
struct dnode *last = NULL;
```



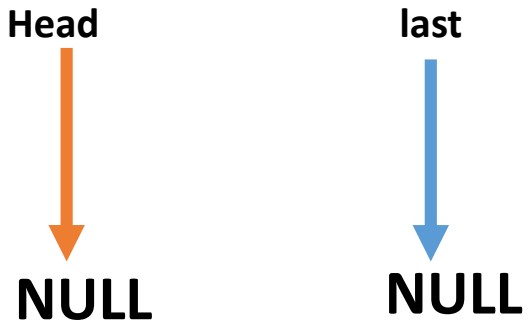
Insert First

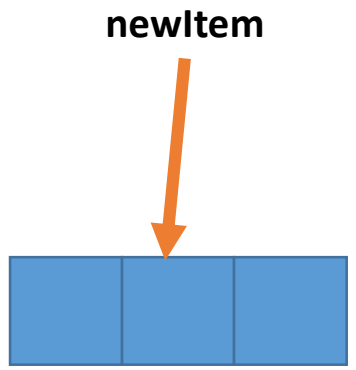
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```



Insert First

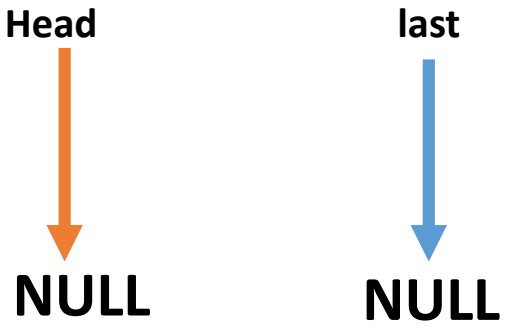
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```

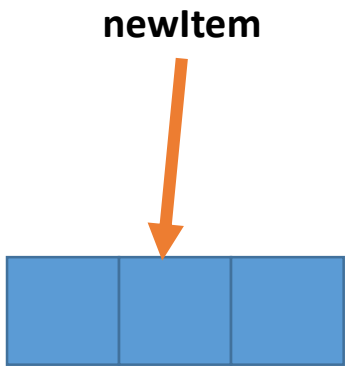




Insert First

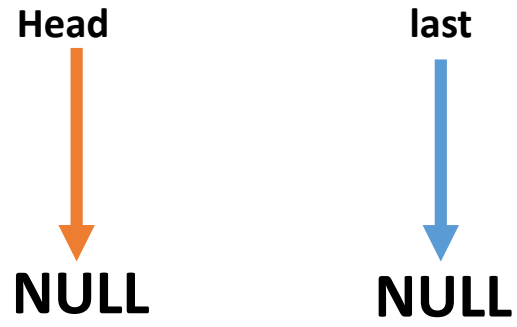
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```

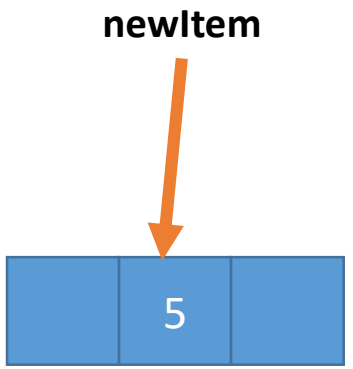




Insert First

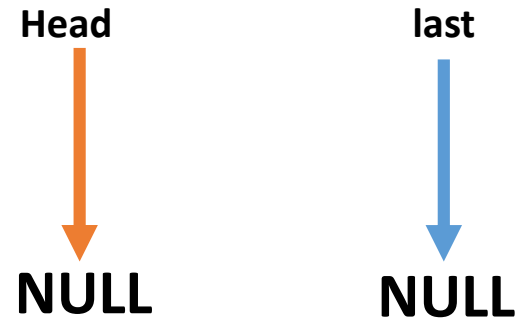
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```

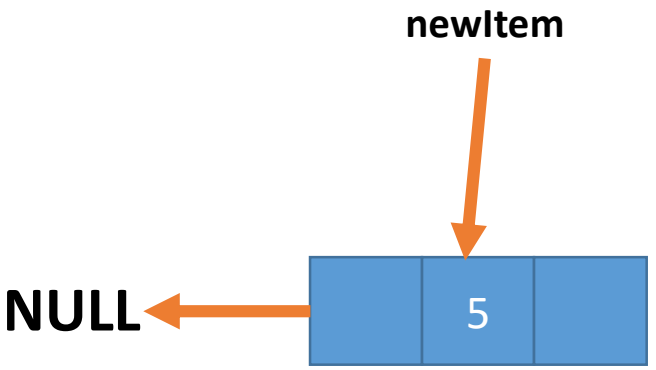




Insert First

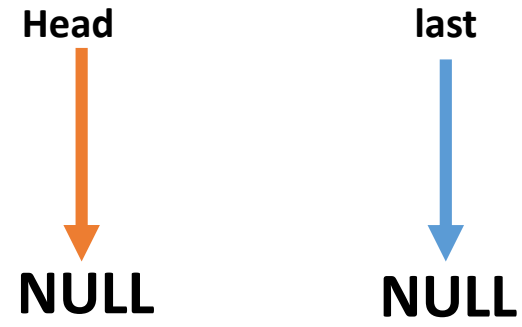
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```

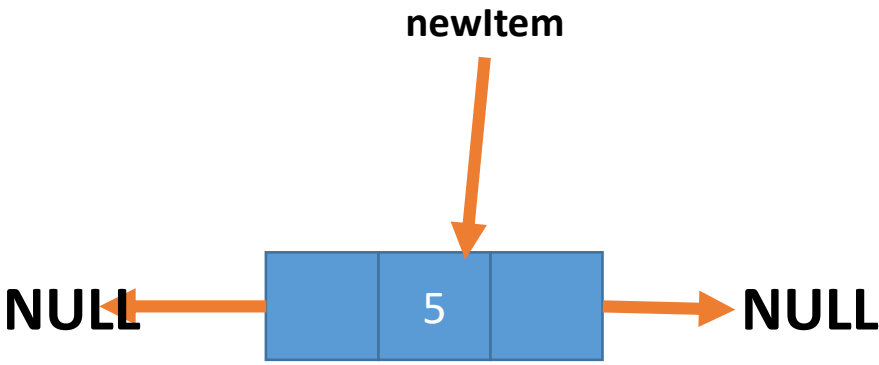




Insert First

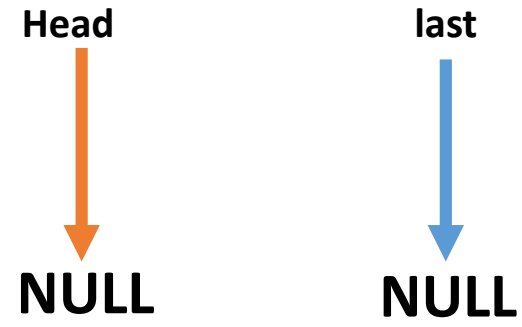
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```

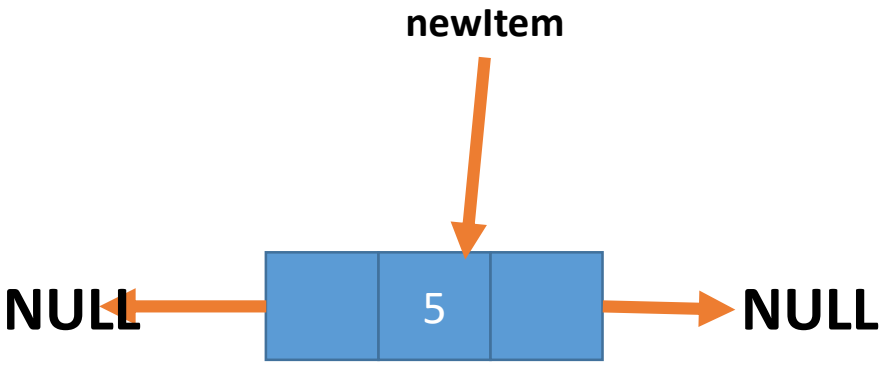




Insert First

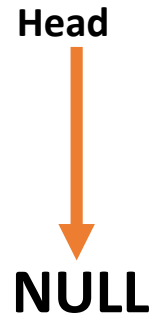
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```

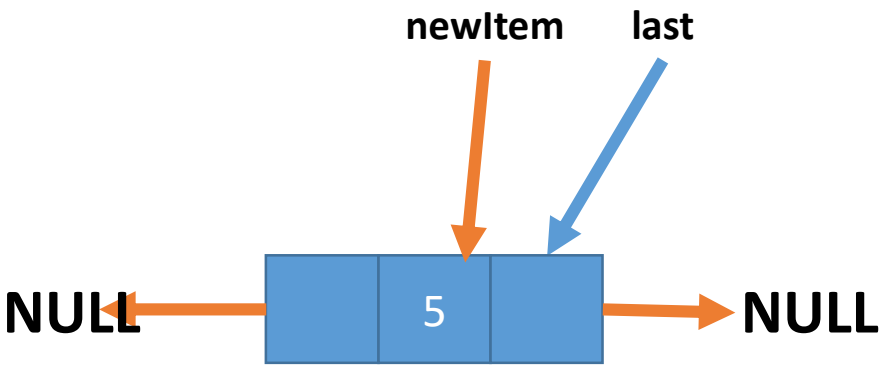




Insert First

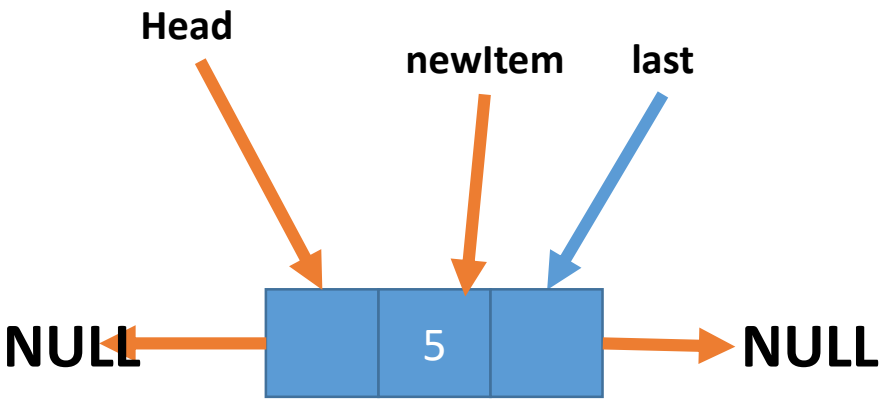
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```





Insert First

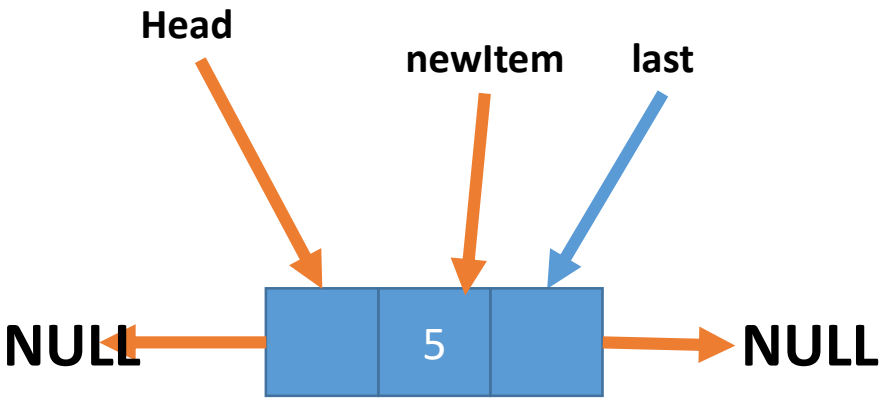
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```



Insert First

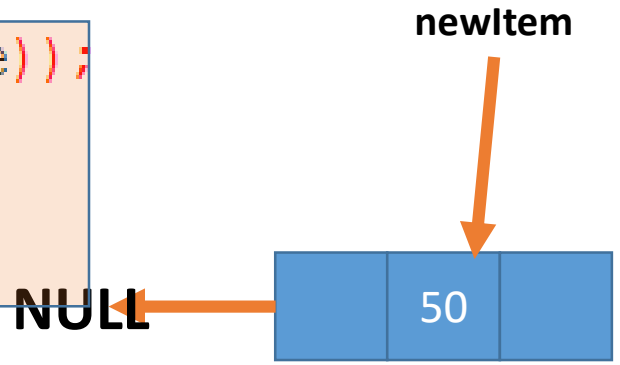
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```

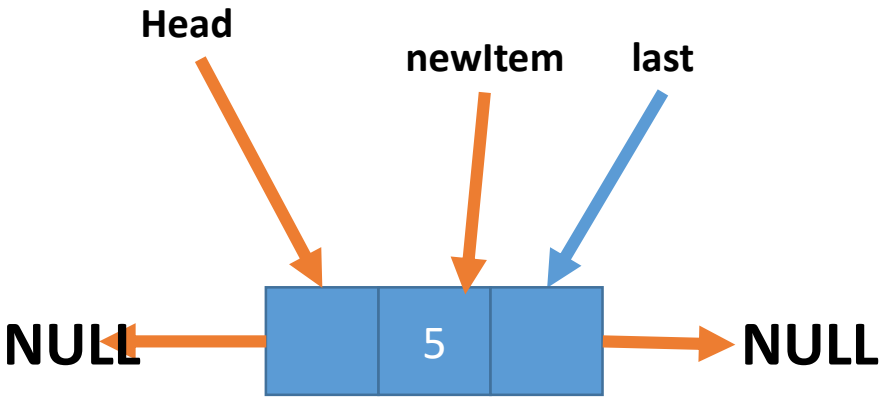
THE LIST IS NO LONGER EMPTY



Insert First

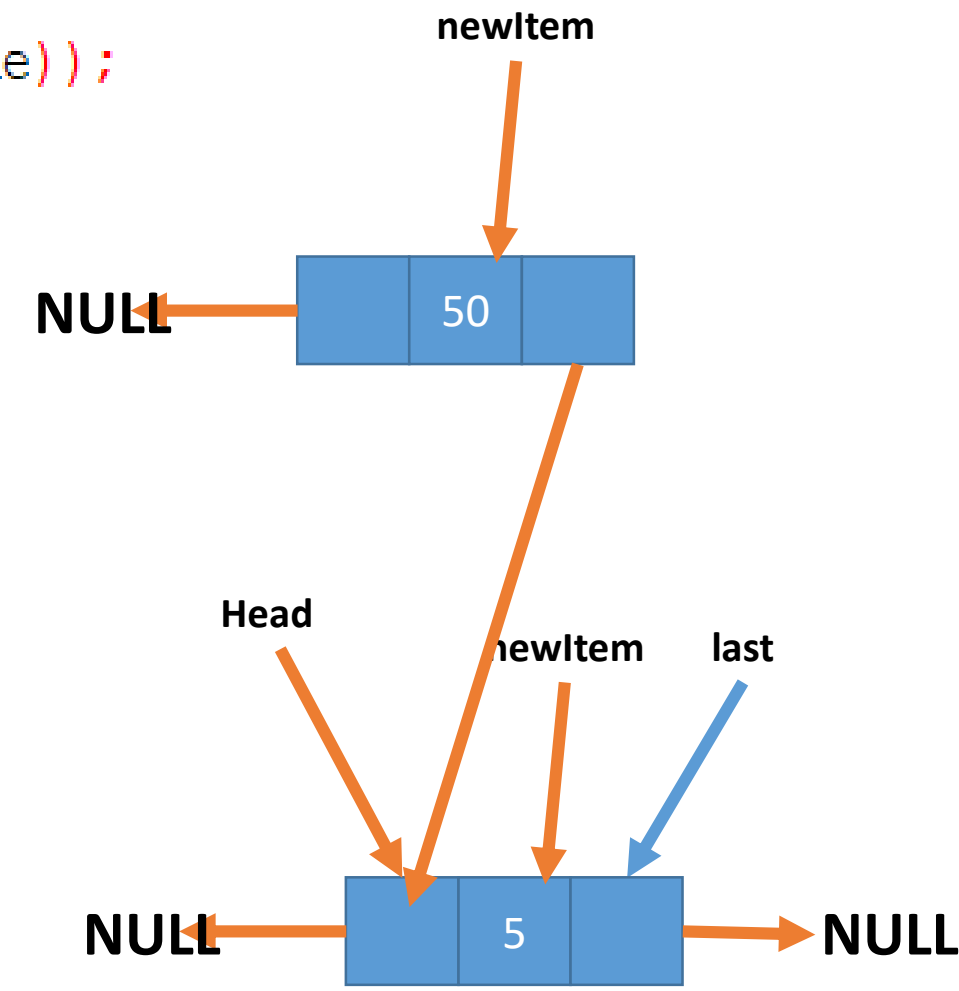
```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```





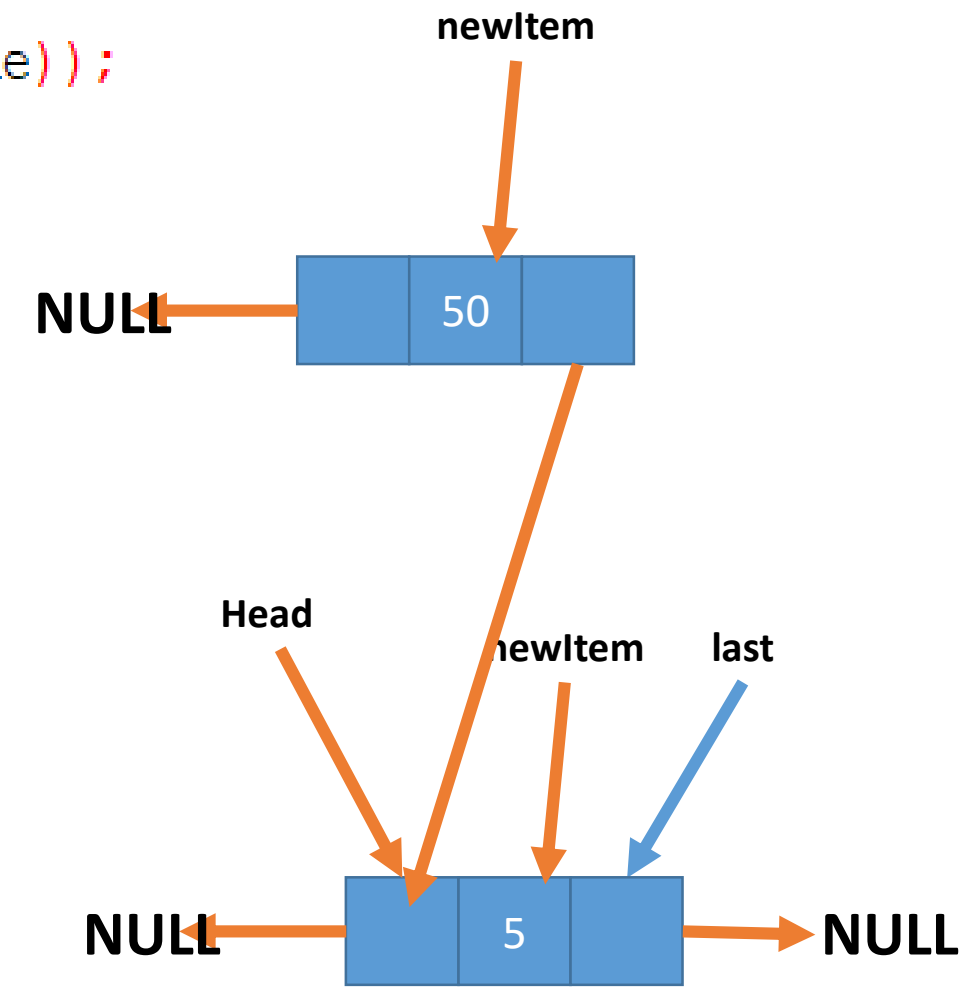
Insert First

```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```



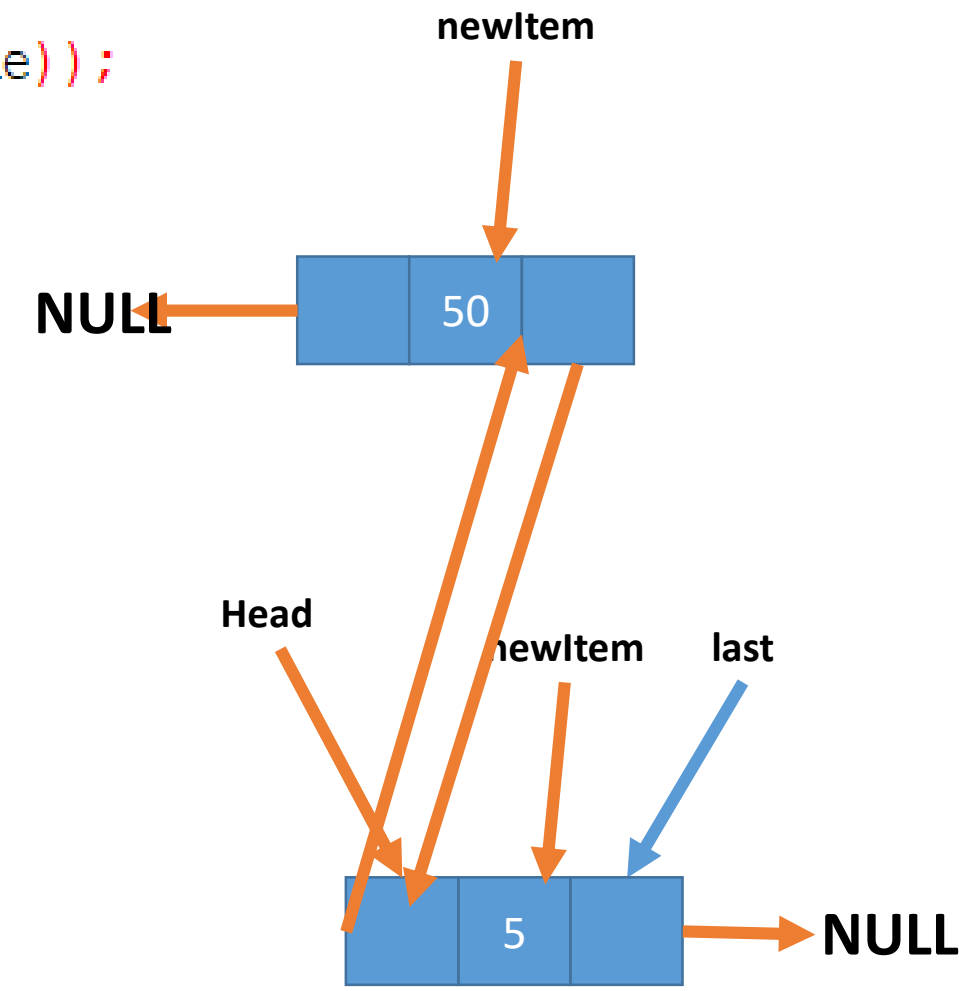
Insert First

```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```



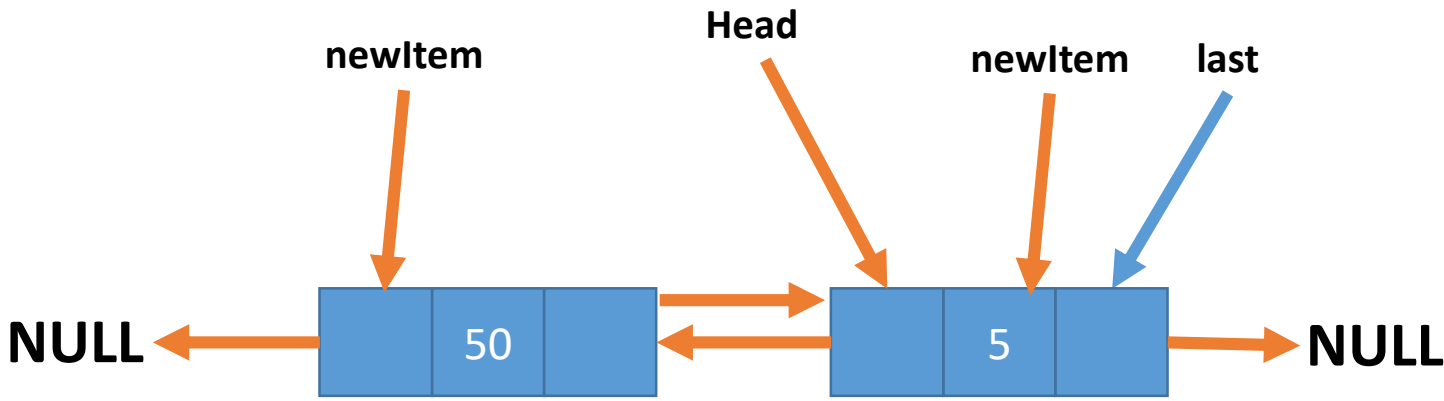
Insert First

```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```



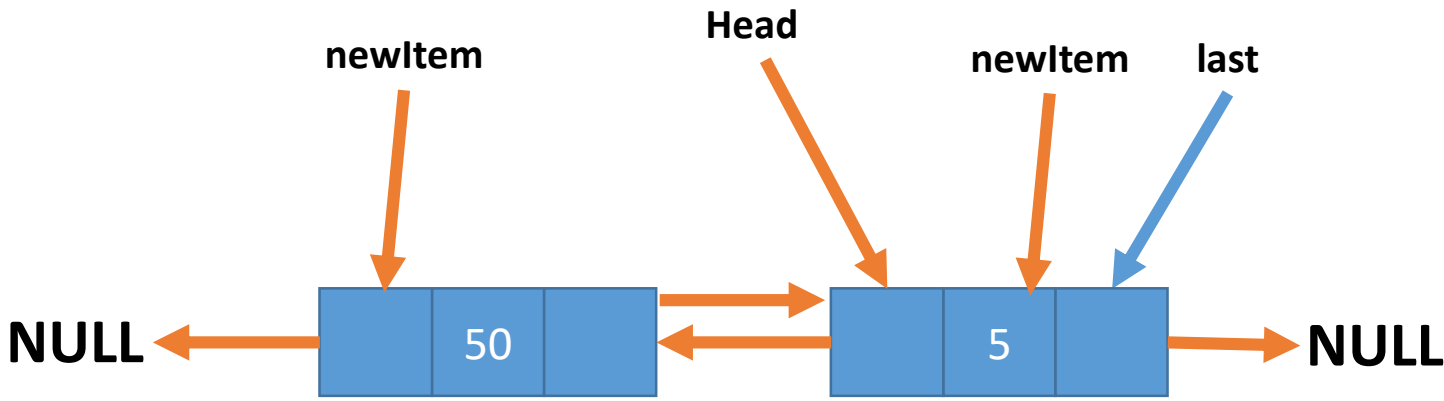
Insert First

```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```



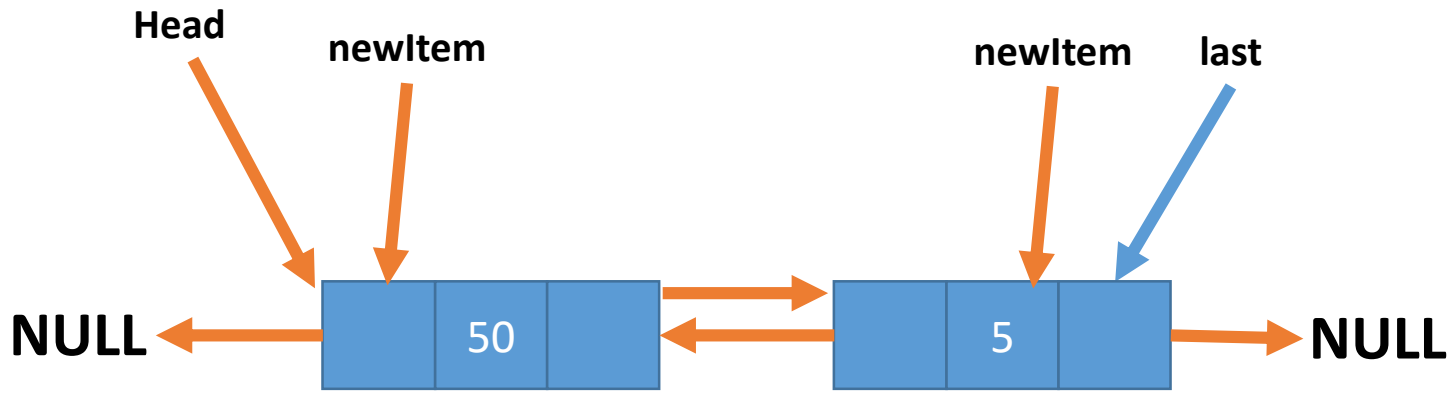
Insert First

```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```



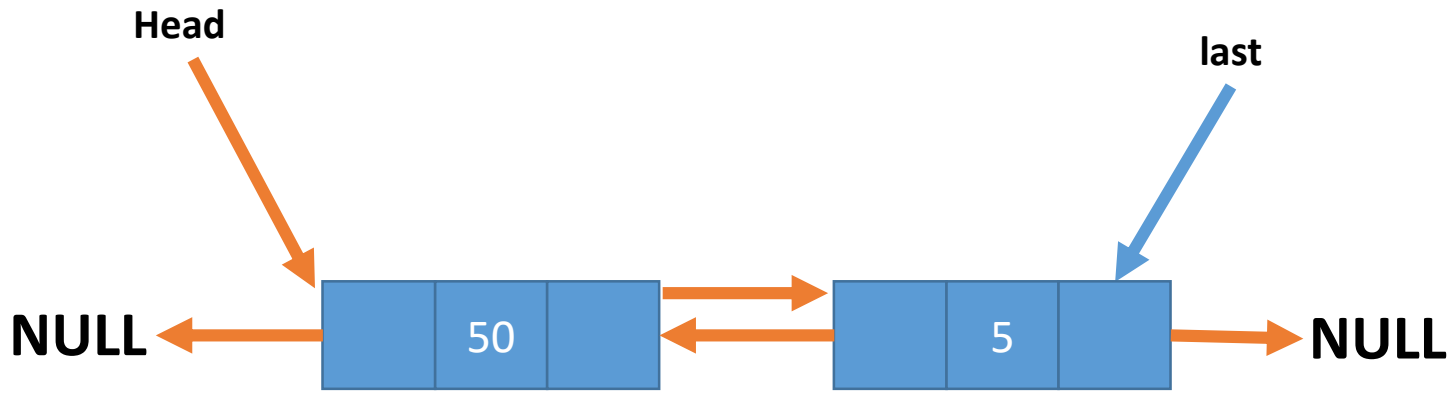
Insert First

```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```



Insert First

```
void insert_beginning(int data){
    struct dnode *newItem = malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = NULL;
    newItem->next = head;

    if(head != NULL){
        head->prev = newItem;
    }
    else{
        last = newItem;
    }
    head = newItem;
}
```



Insert Last

- Try writing the function yourself.
  - Consider the case where the list is empty
  - Consider the case where the list is not empty
  - Be sure to deal with all the pointers

Insert Last

```
void insert_end(int data){
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));

    newItem->value = data;
    newItem->prev = last;
    newItem->next = NULL;

    if(head == NULL){
        head = newItem;
        last = newItem;
    }
    else{
        last->next = newItem;
        last = newItem;
    }
}
```

Search Function

- We will this function to search for a particular node.
- The function will return a pointer to that node.
- Try it yourself

Search Function

- We will this function to search for a particular node.
- The function will return a pointer to that node.

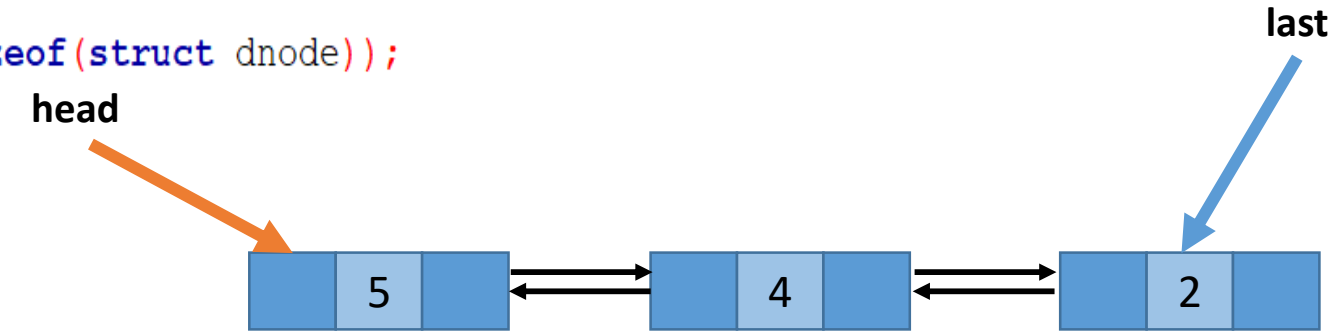
```
struct dnode* search_list(int data){
    struct dnode *currentNode = head;

    while(currentNode != NULL){
        if(currentNode->value == data){
            return currentNode;
        }
        currentNode = currentNode->next;
    }

    return NULL;
}
```

Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```

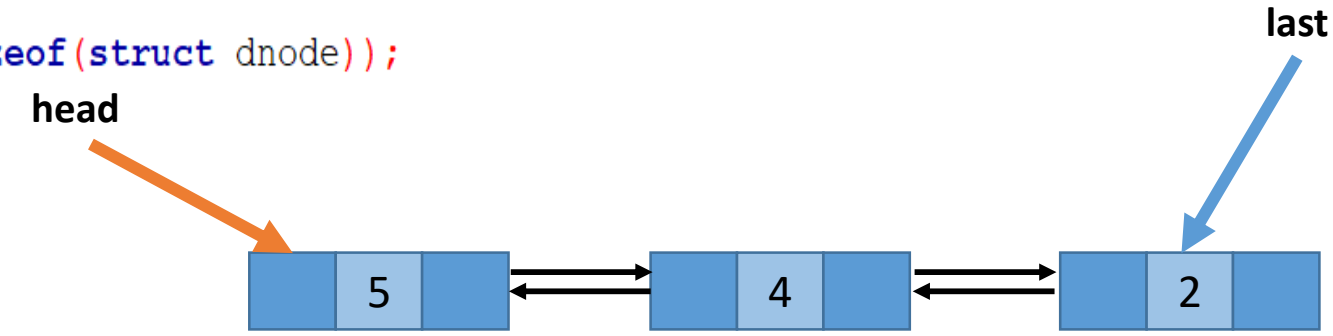
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



Insert After Function

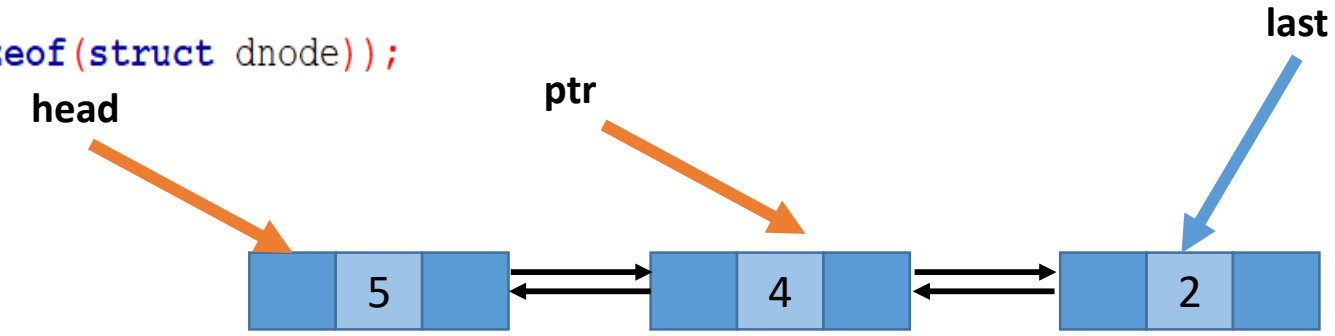
```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



Search(4);

Insert After Function

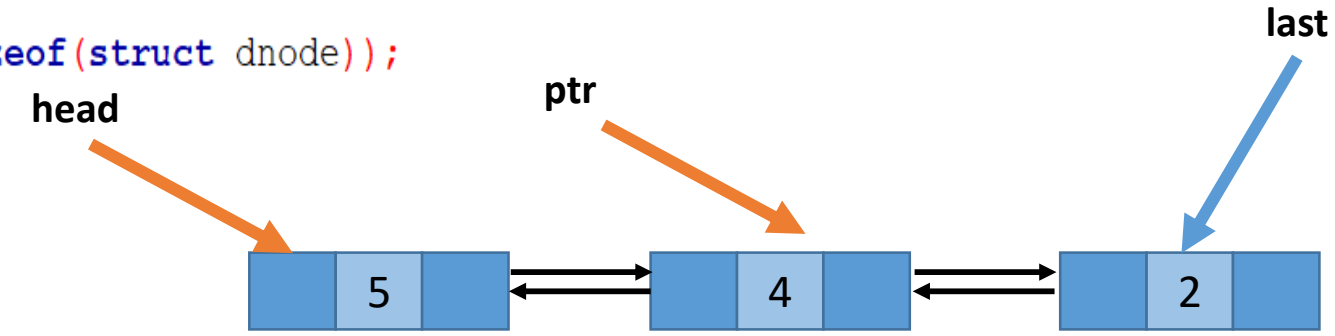
```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



ptr=Search(4);

Insert After Function

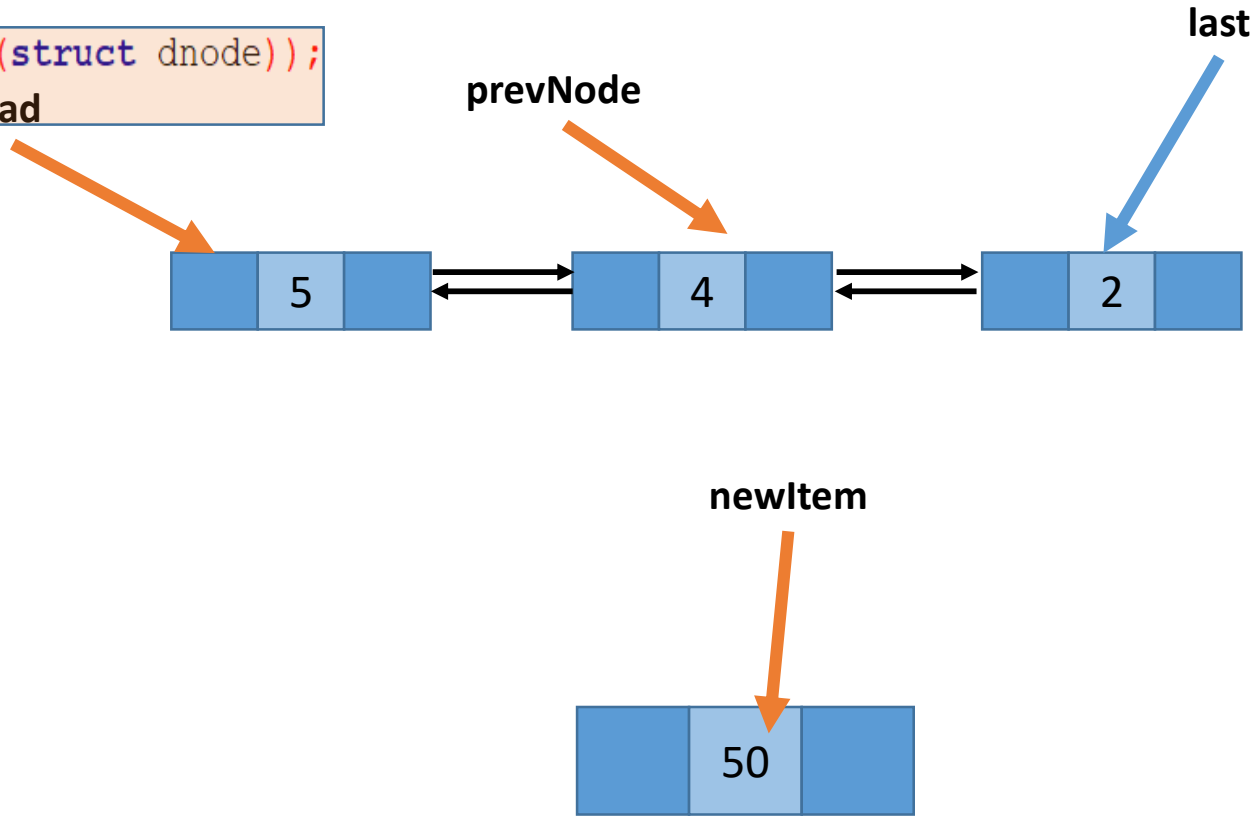
```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



ptr=Search(4);
Insert_after_node(50,ptr);

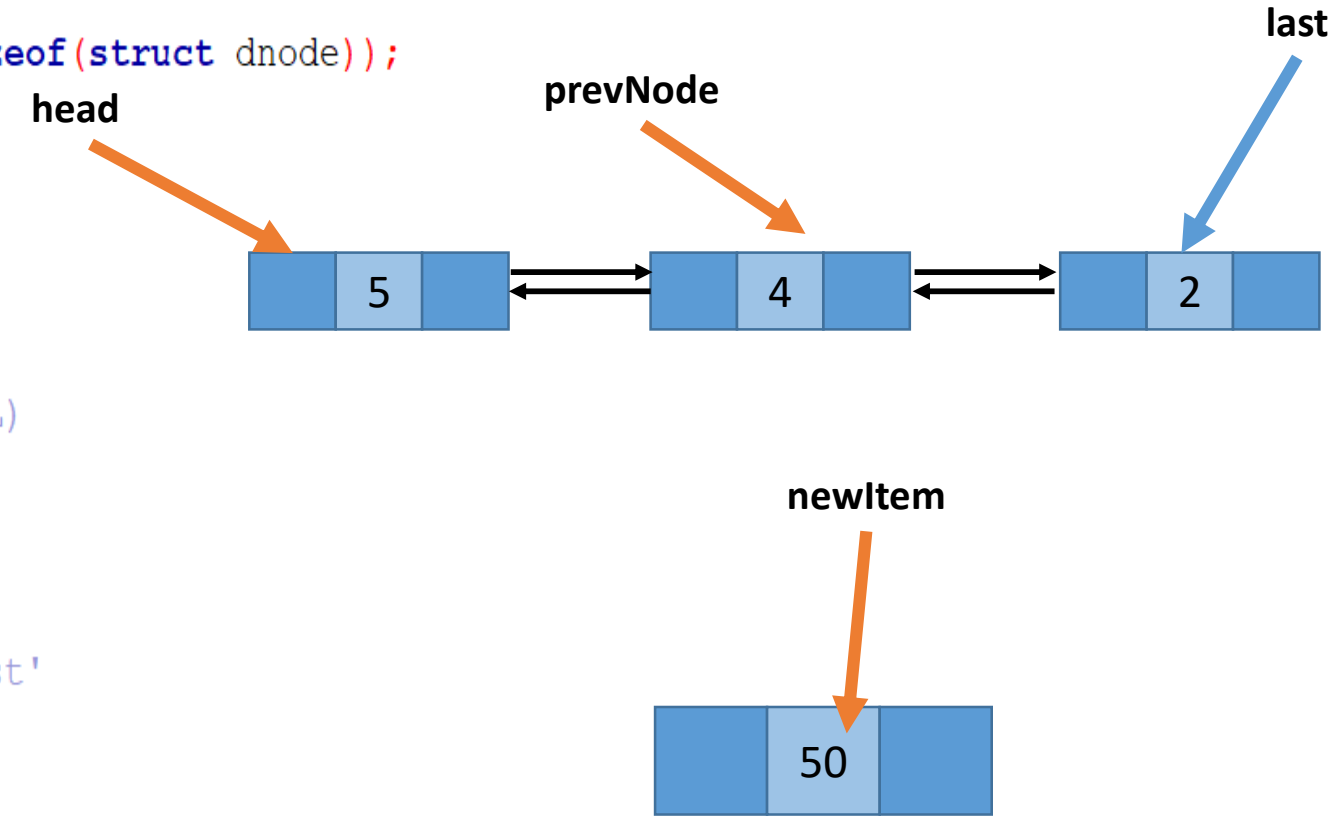
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



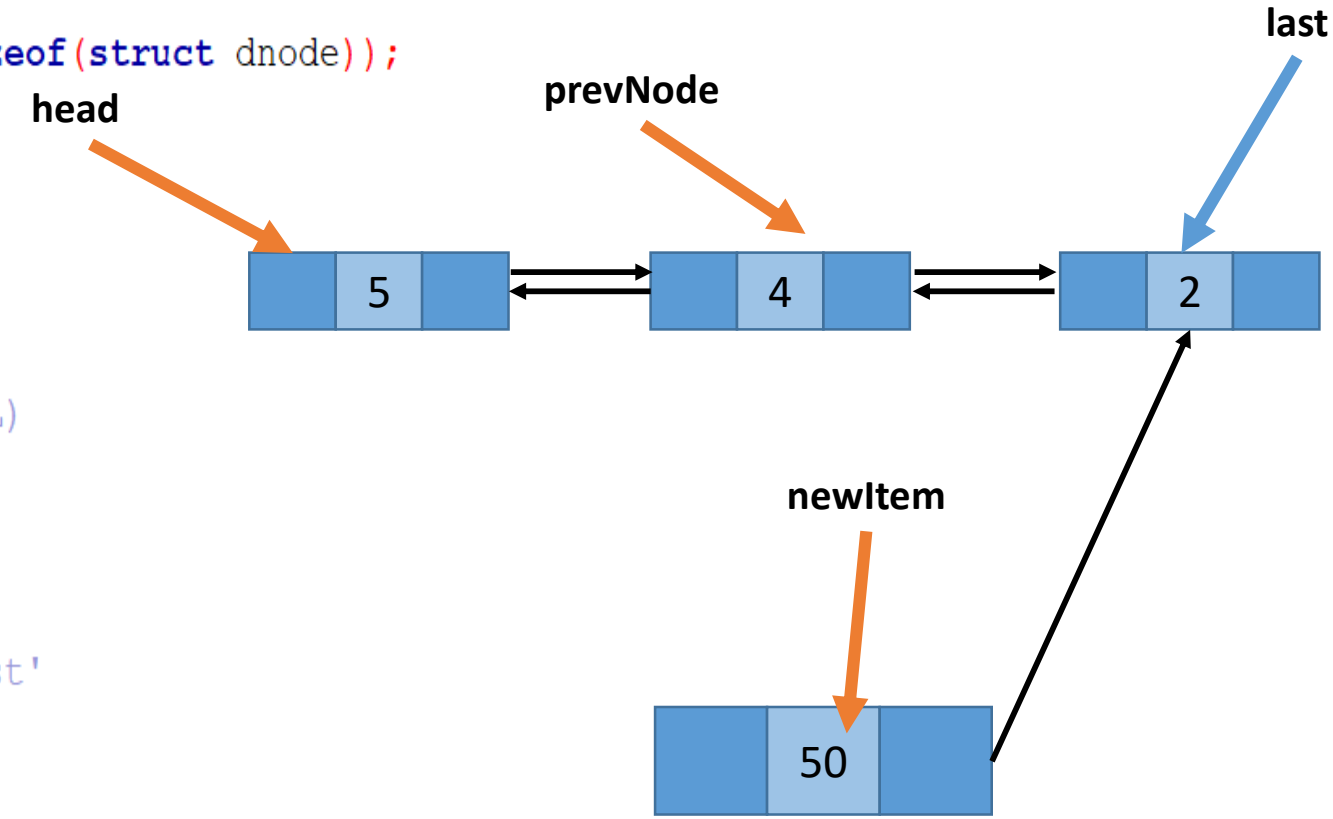
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



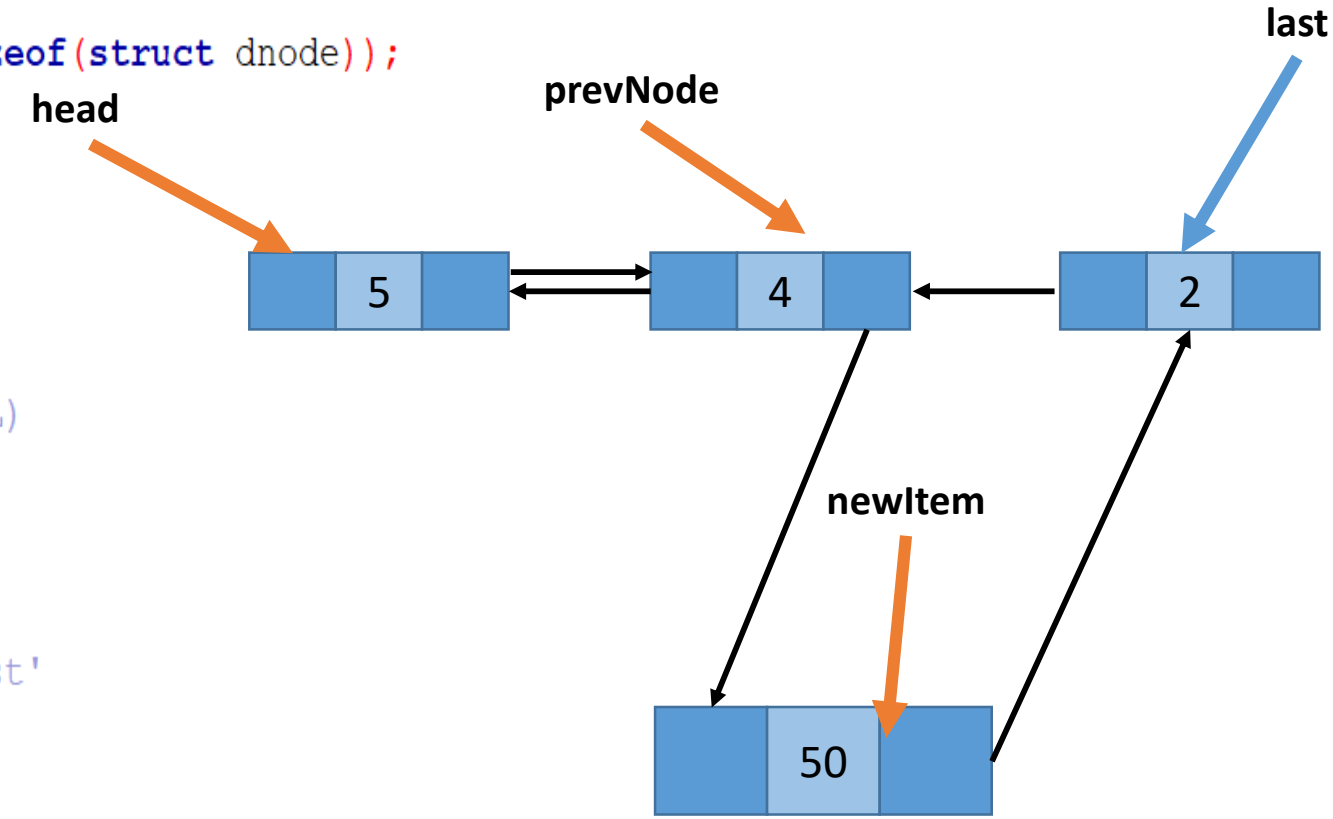
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



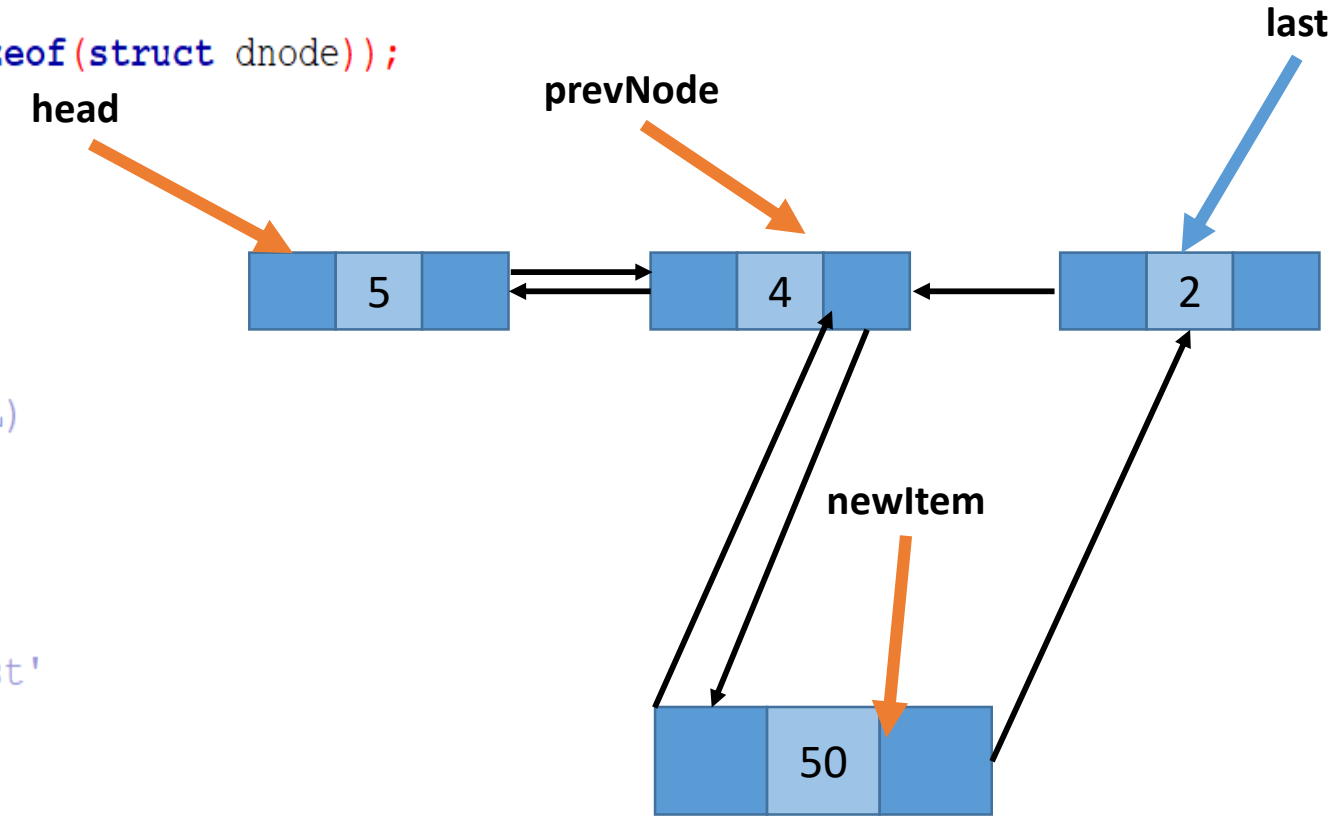
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



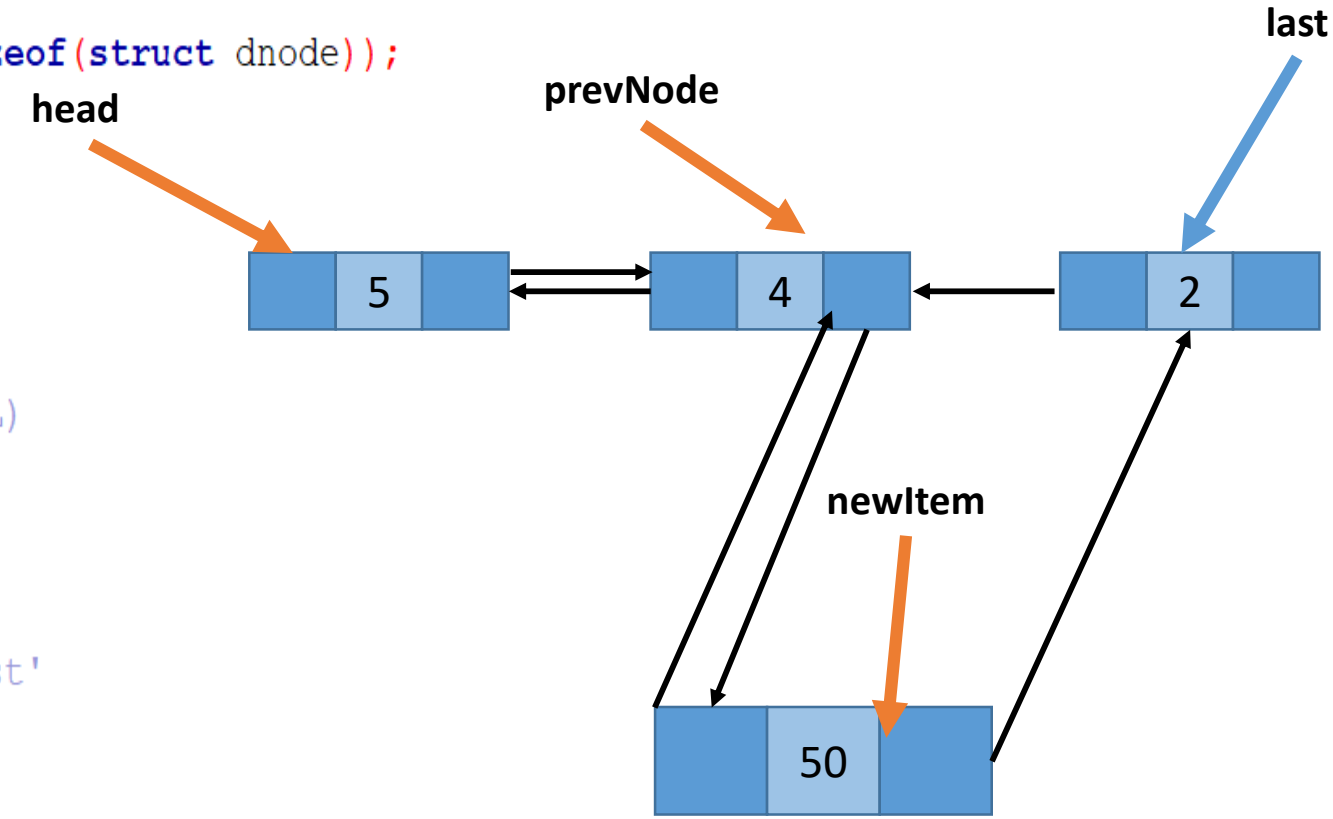
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



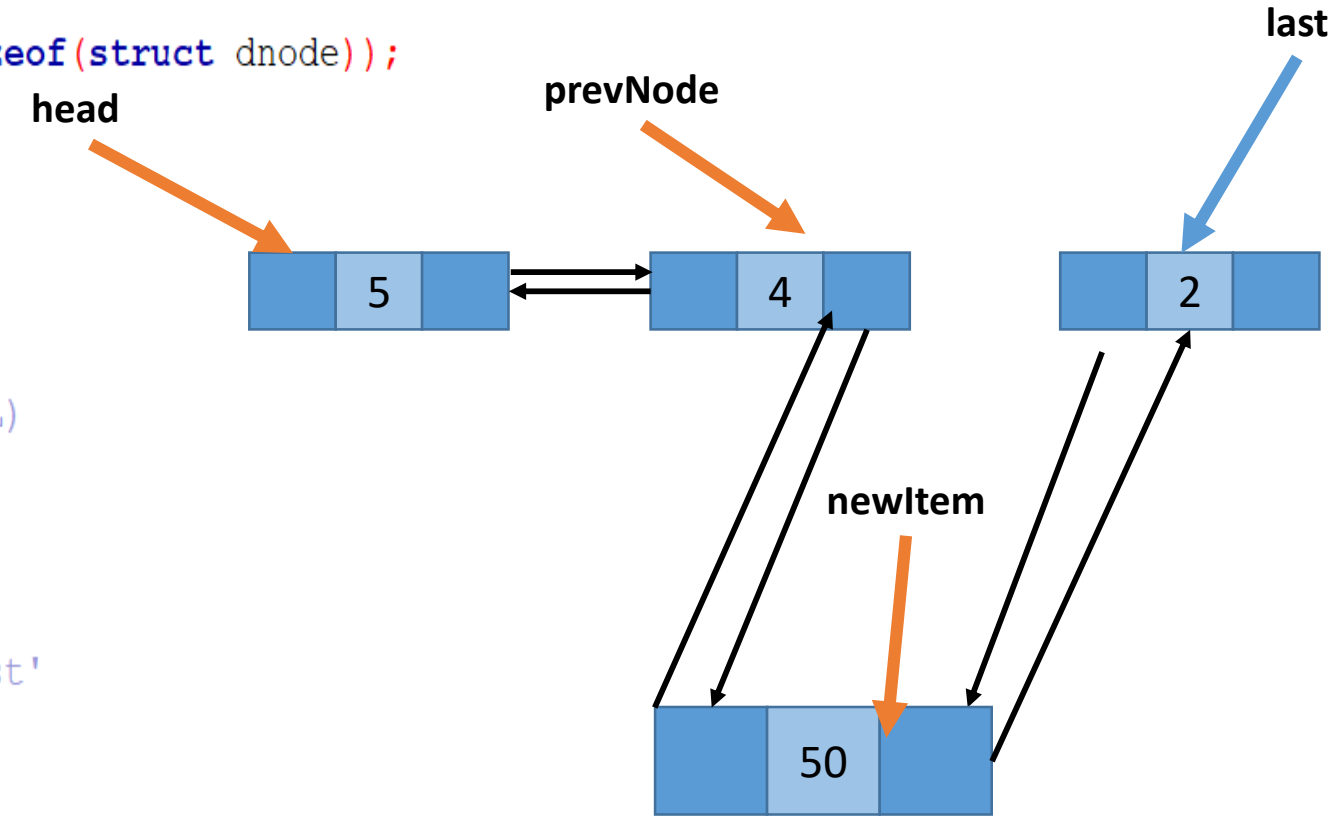
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



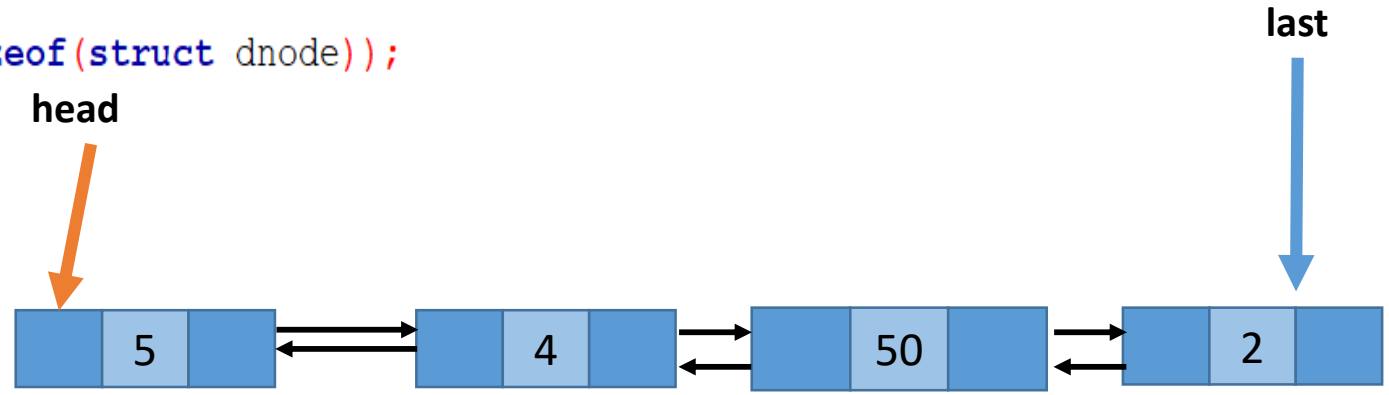
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



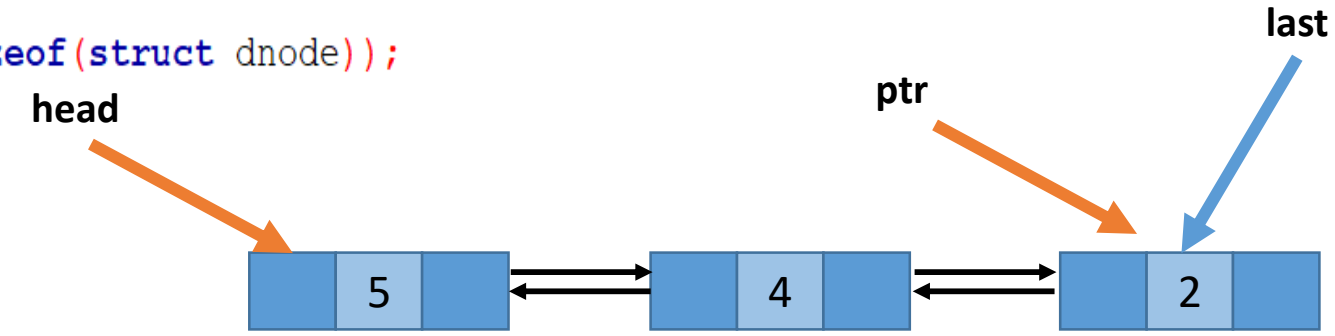
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



Insert After Function

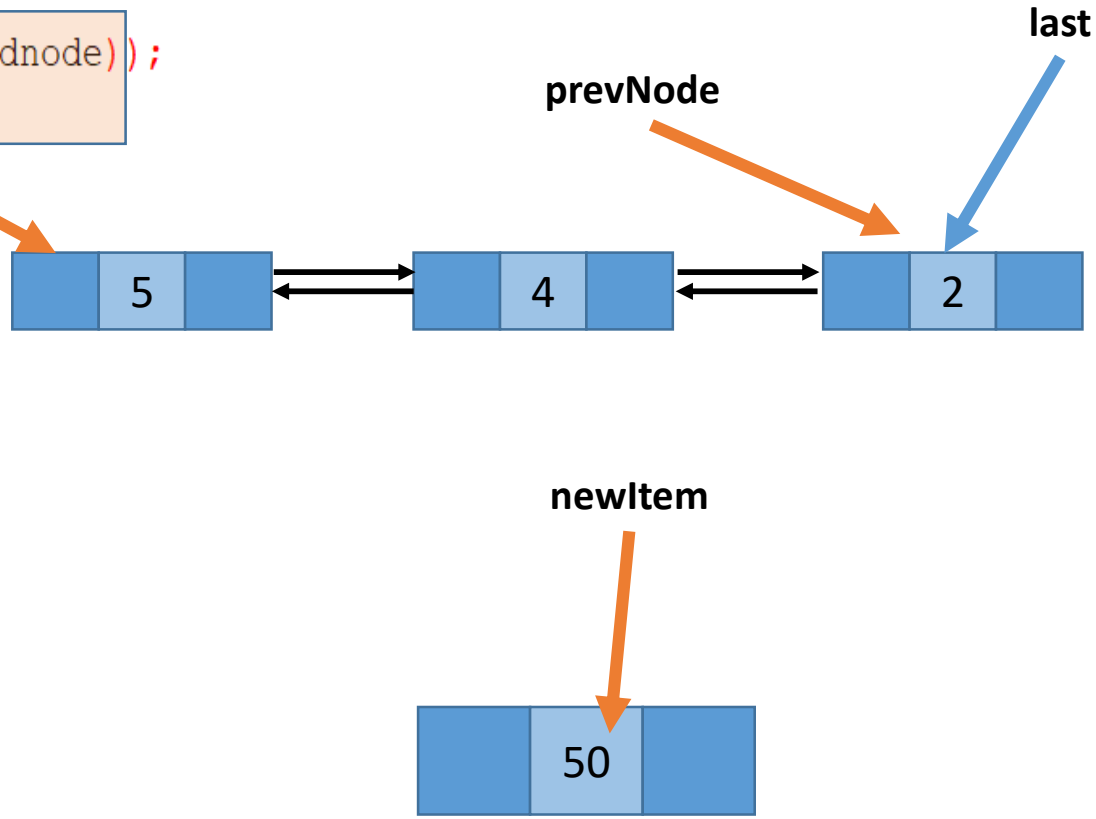
```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



ptr=Search(2);
Insert_after_node(50,ptr);

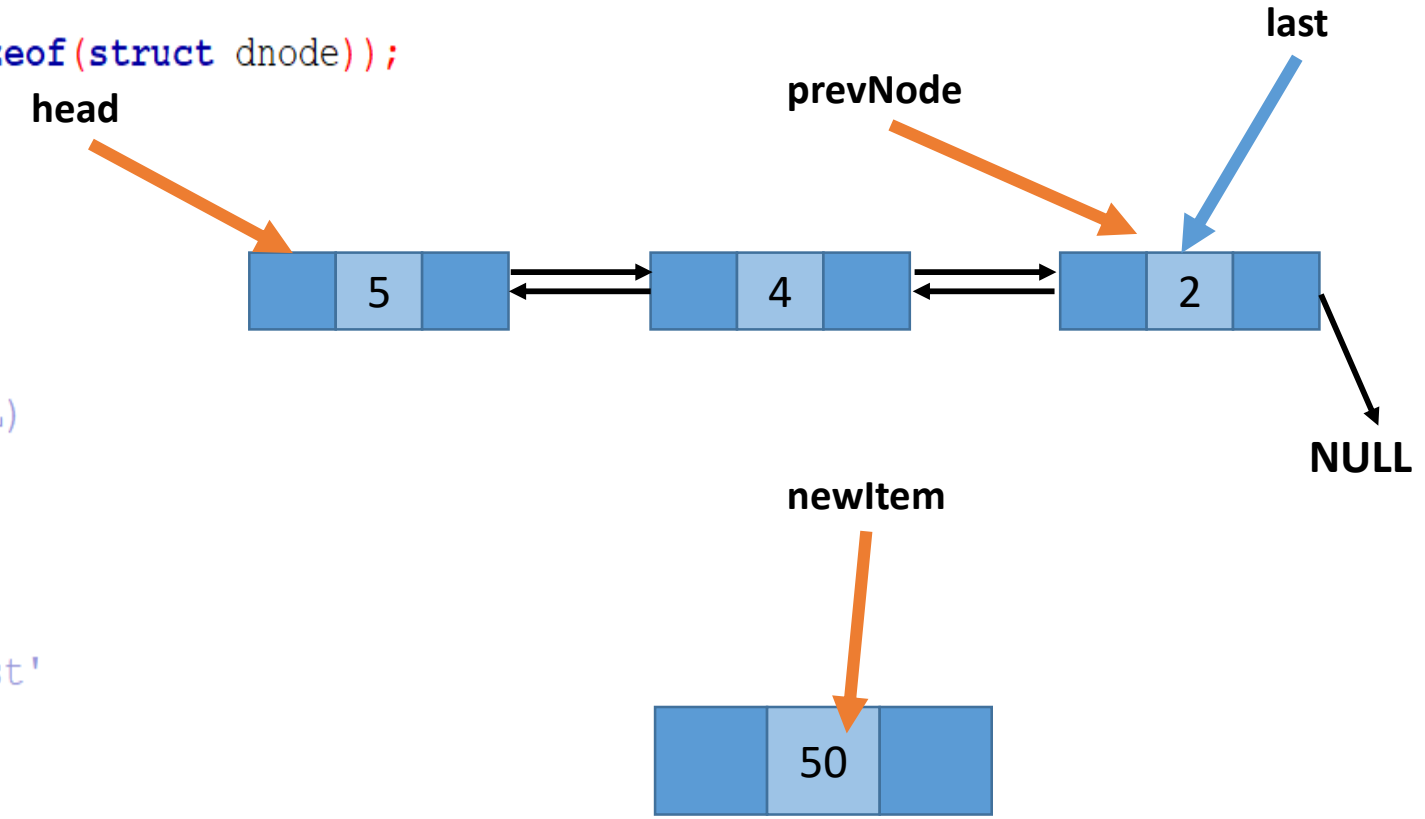
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



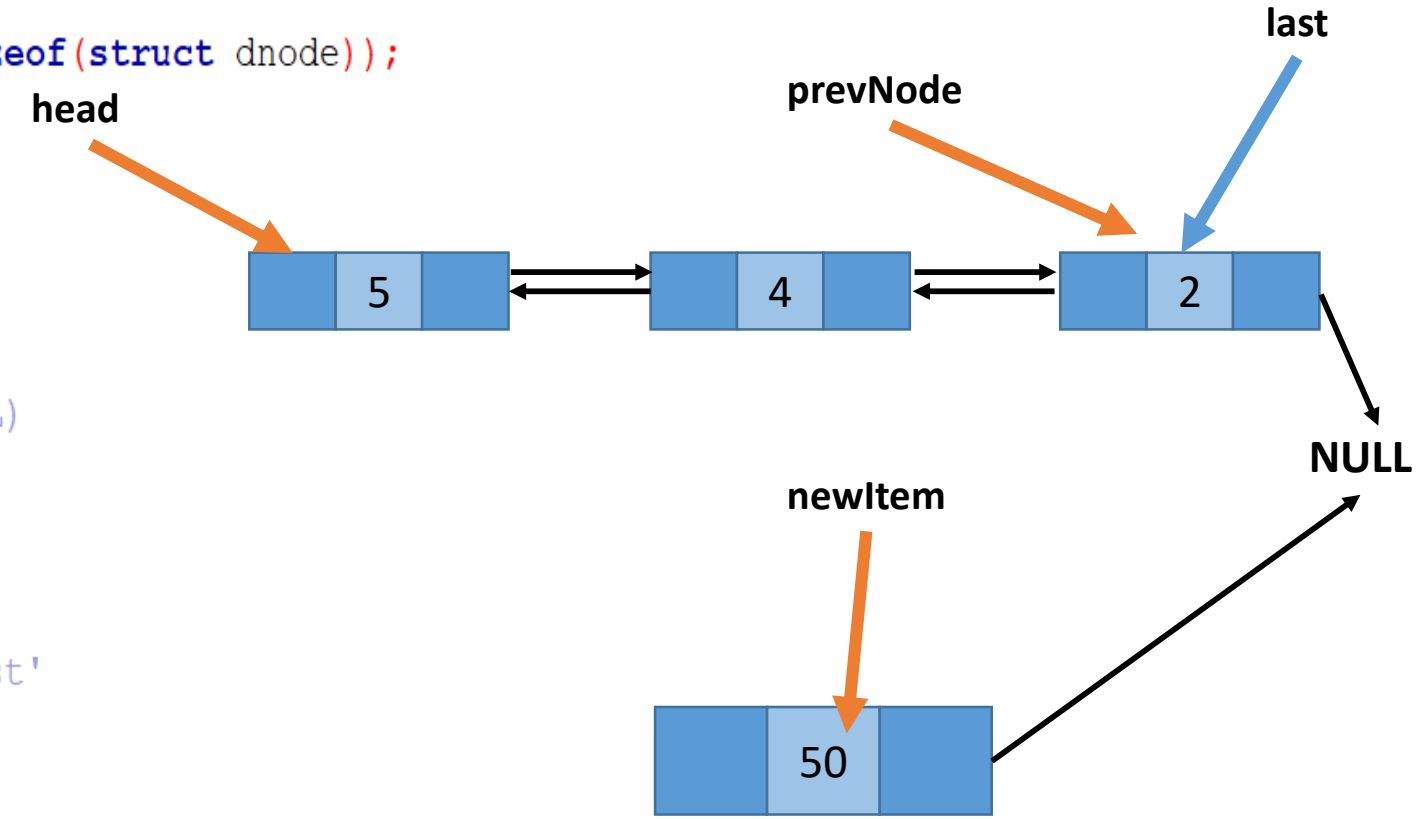
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



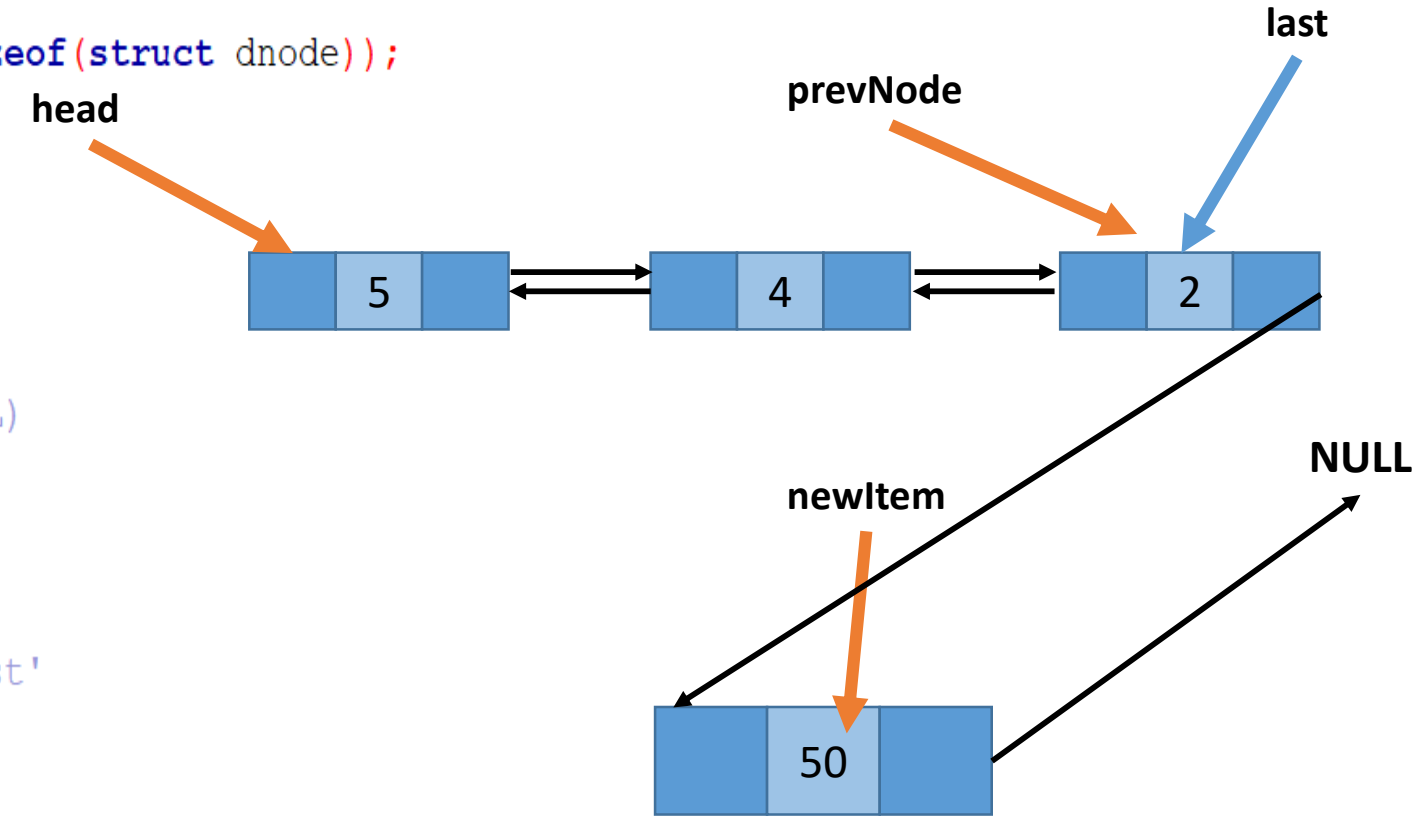
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



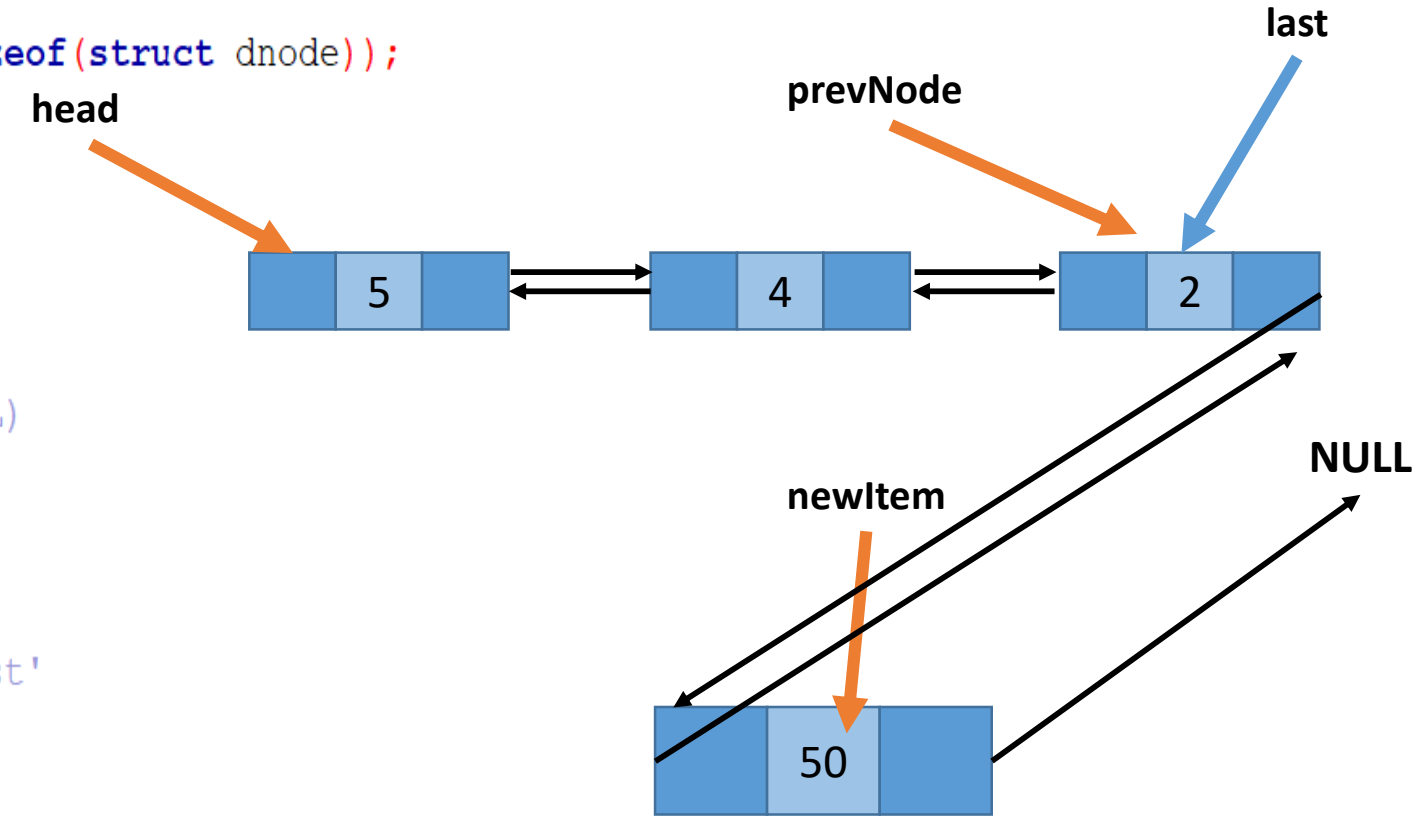
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



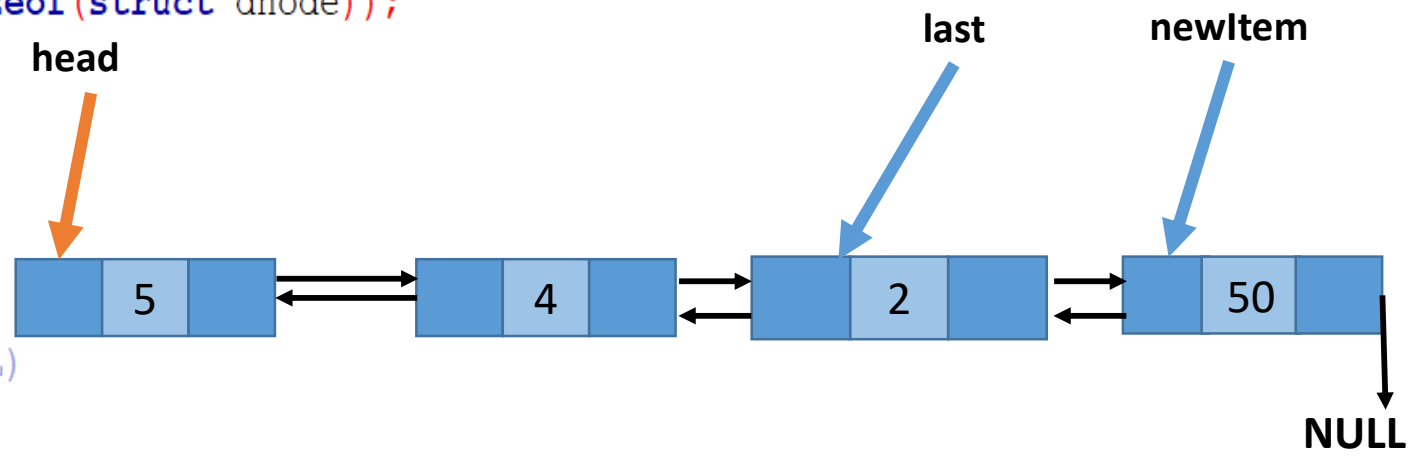
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



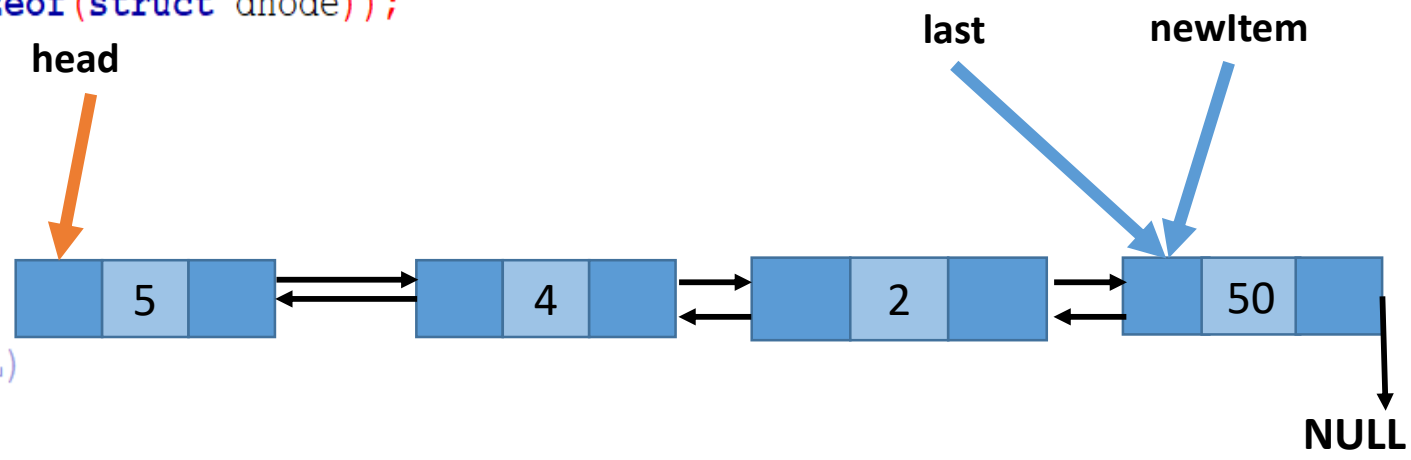
Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



Insert After Function

```
void insert_after_node(int data, struct dnode* prevNode){
    if(prevNode == NULL){
        printf("Previous node cannot be NULL\n");
        return;
    }
    struct dnode *newItem = (struct dnode *)malloc(sizeof(struct dnode));
    newItem->value = data;

    newItem->next = prevNode->next;
    prevNode->next = newItem;
    newItem->prev = prevNode;

    // Update previous of new node's next (if not NULL)
    if(newItem->next != NULL){
        newItem->next->prev = newItem;
    }

    // If the node was inserted at the end update 'last'
    if(newItem->next == NULL){
        last = newItem;
    }
}
```



Deletion

- Deletion is to remove a node from a list. It can also take place anywhere -- the first, last, or interior of a linked list.
- To delete a node from a double linked list is easier than to delete a node from a linear linked list.
- For deletion of a node in a single linked list, we have to search and find the predecessor of the discarded node.
- But in the double linked list, no such search is required.
- Given the **address** (obtained from the **search function**) of the node that is to be deleted, the predecessor and successor nodes are immediately known

Deletion

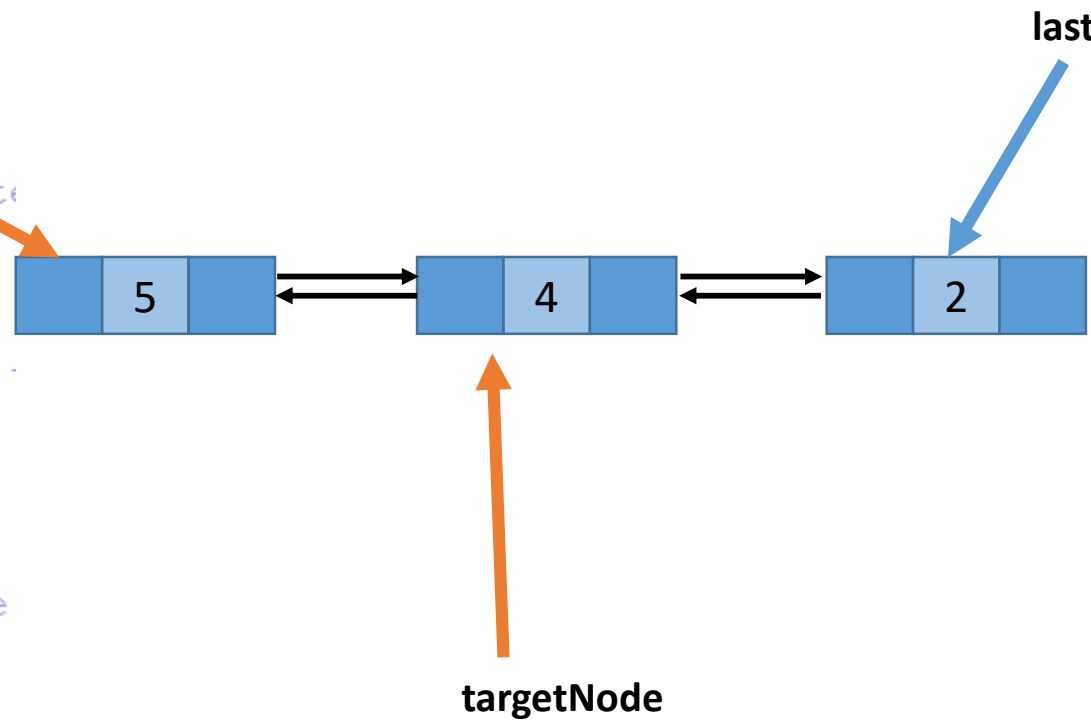
- First, let us check if the provided node is NULL.
- Next, let us check if the node is the first in the list (i.e., it has no previous node).
  - If the node to be deleted **is the first node**, it sets the head of the list to be the next node.
  - If **it's not the first node**, the function sets the next pointer of the previous node to the node after the node to be deleted, effectively skipping over the node to be deleted in the list.
- Then we need to check whether there is a node **after the node to be deleted**.
- Finally, we need to free the target node from the memory.

Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to updat
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

Delete A Target Node

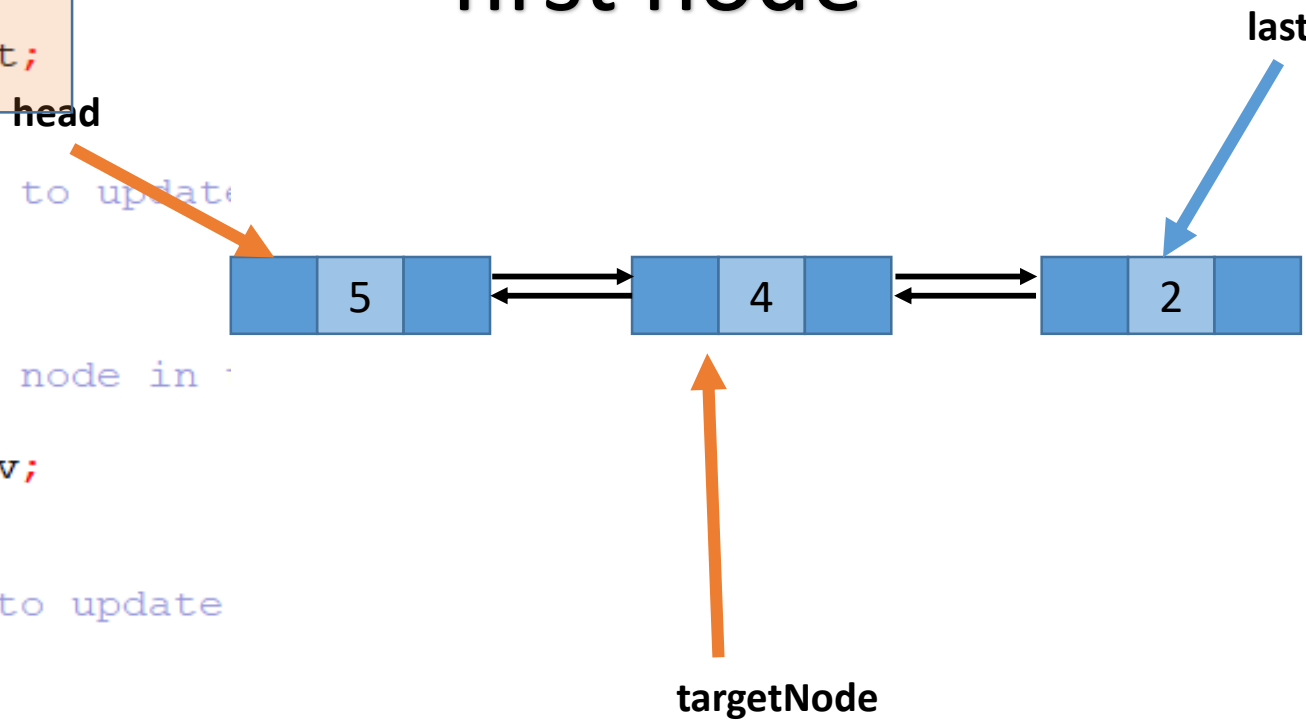
```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to update
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

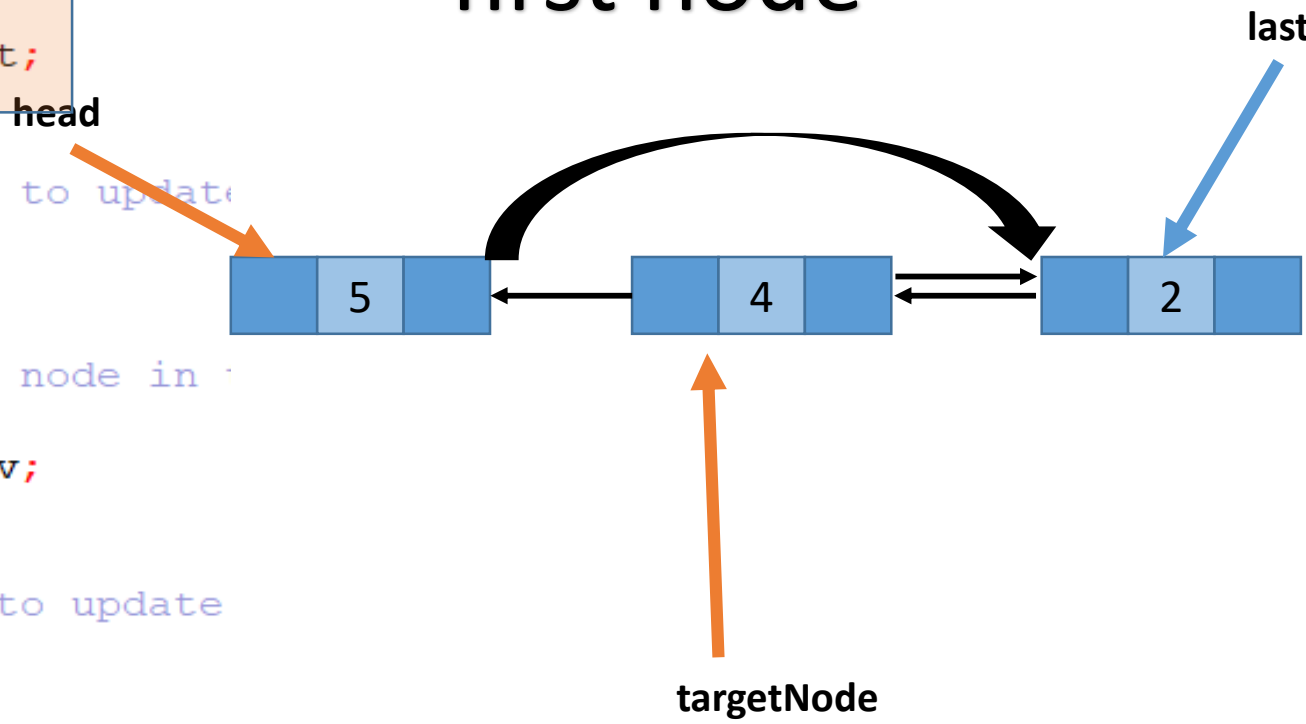
Target Case 1: It is not the first node



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to update
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

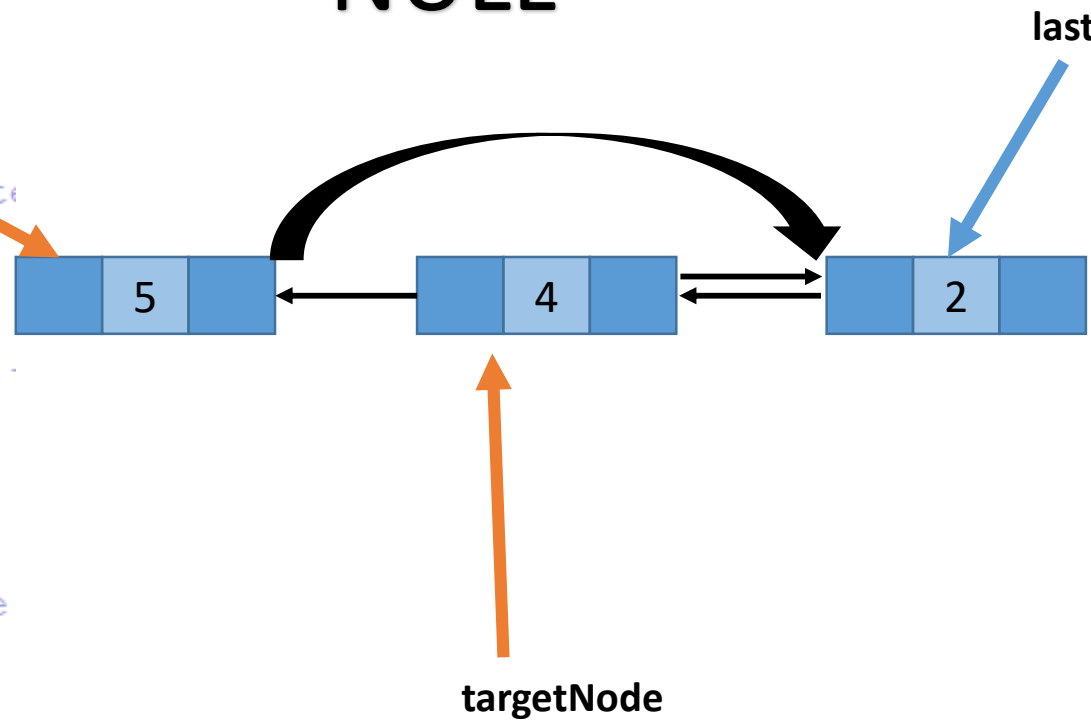
Target Case 1: It is not the first node



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

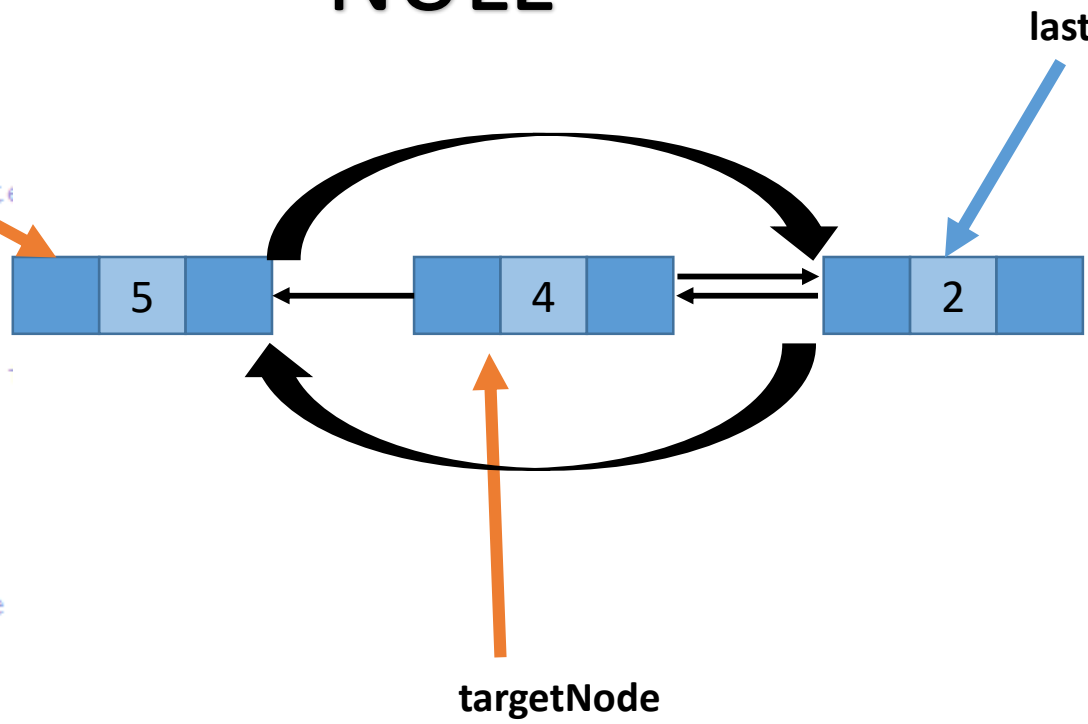
Successor Case 1: It is not NULL



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

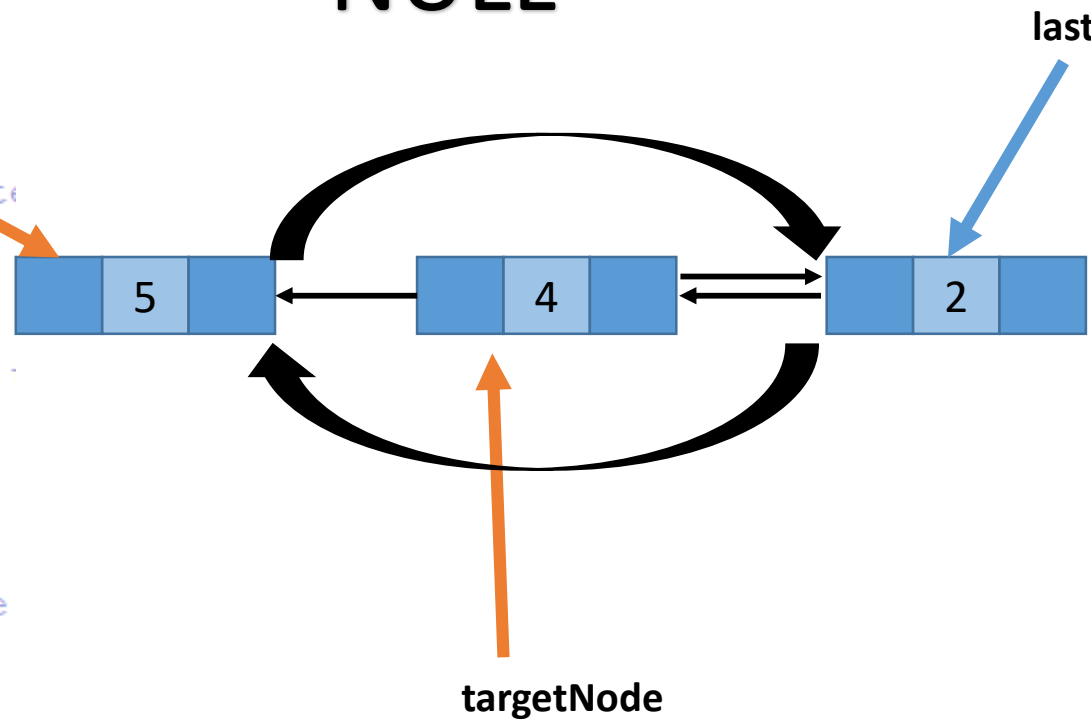
Successor Case 1: It is not NULL



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

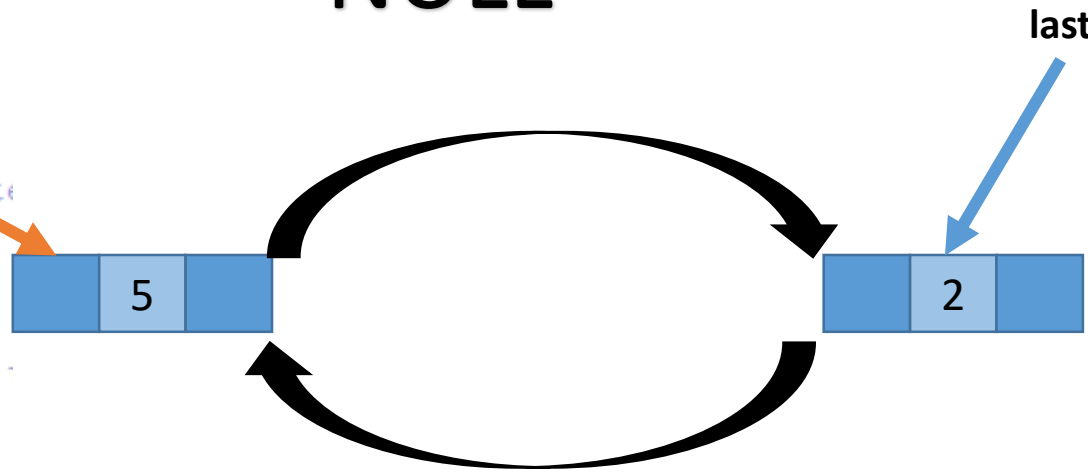
Successor Case 1: It is not NULL



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

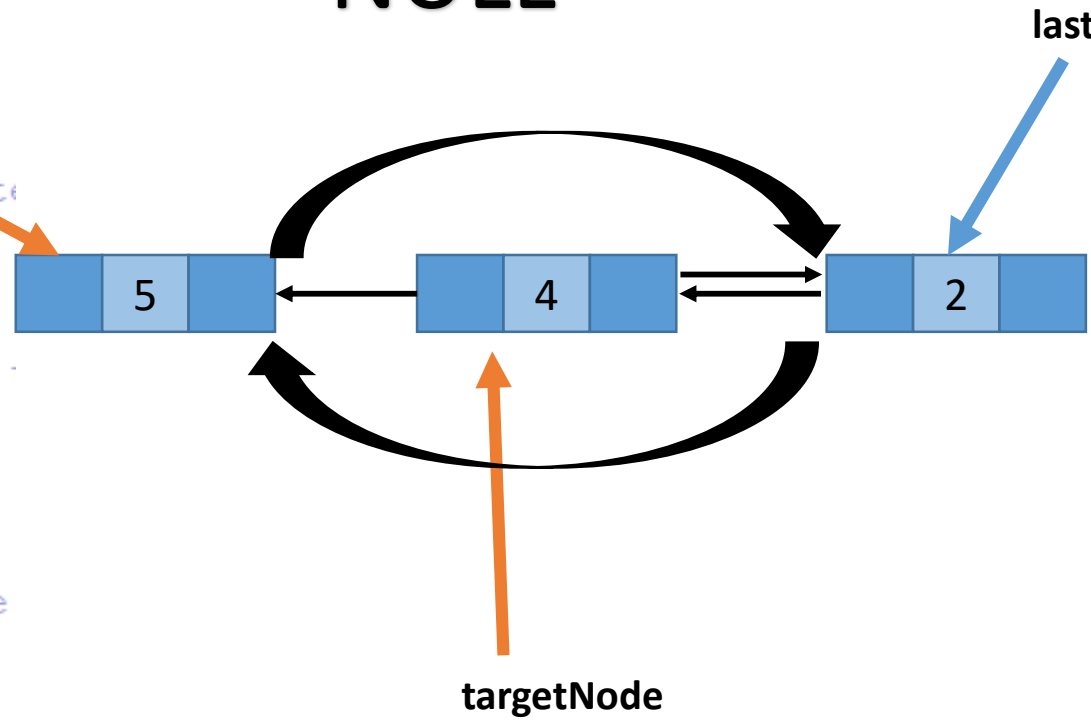
Successor Case 1: It is not NULL



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

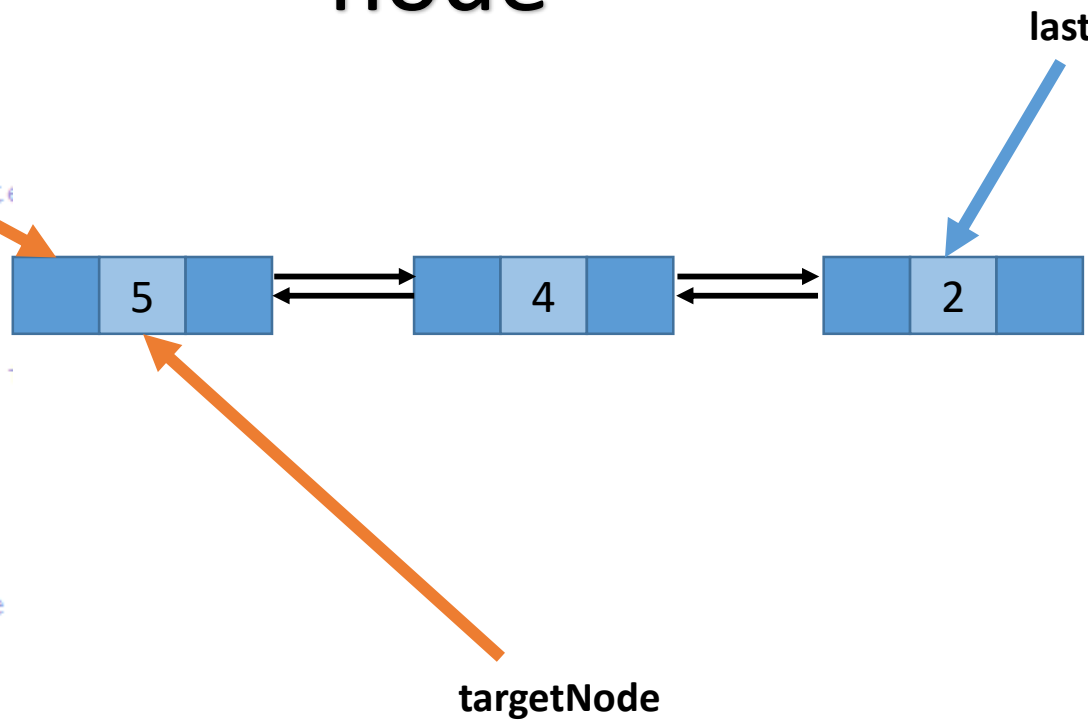
Successor Case 1: It is not NULL



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

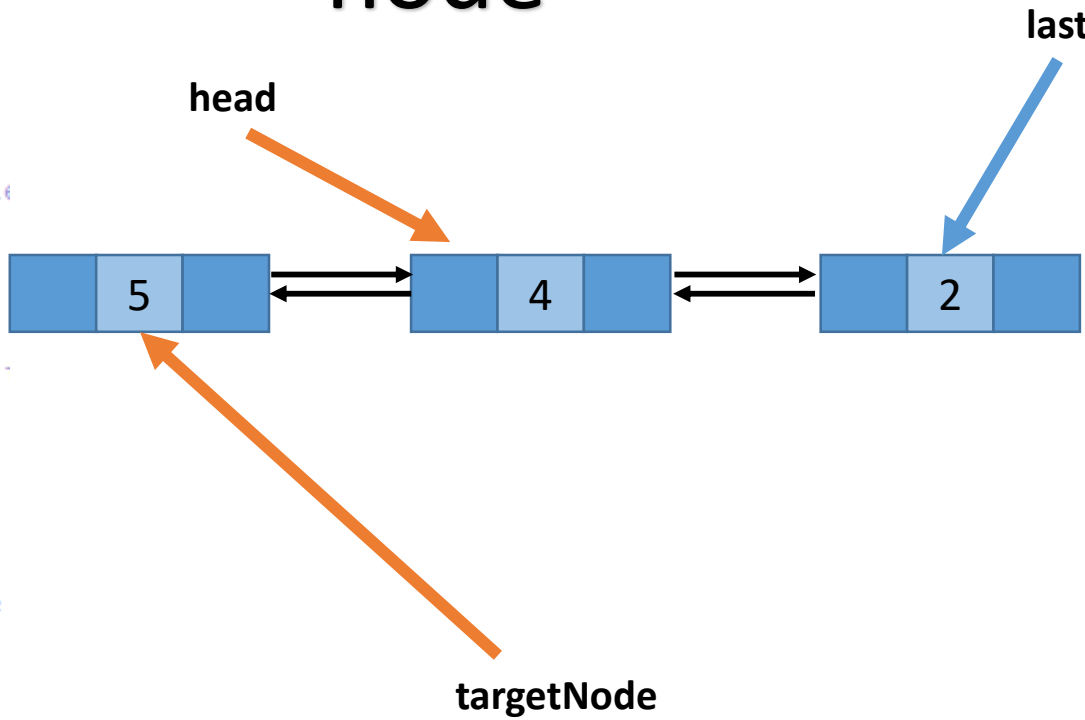
Target Case 2: It is the first node



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to updat
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

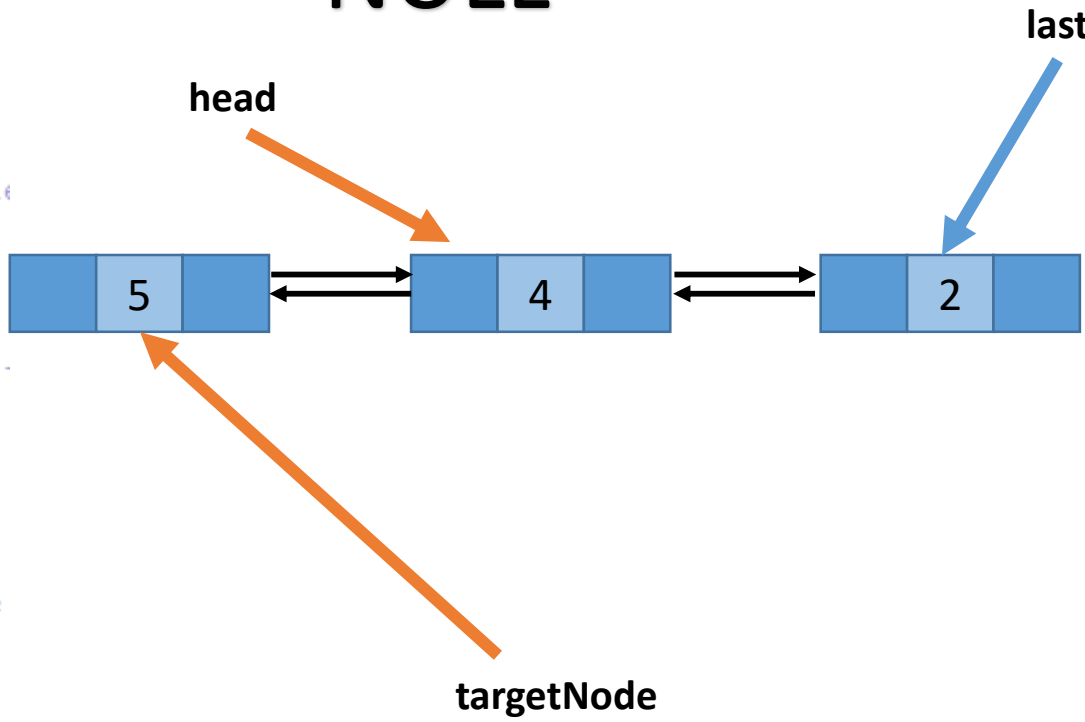
Target Case 2: It is the first node



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to updat
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

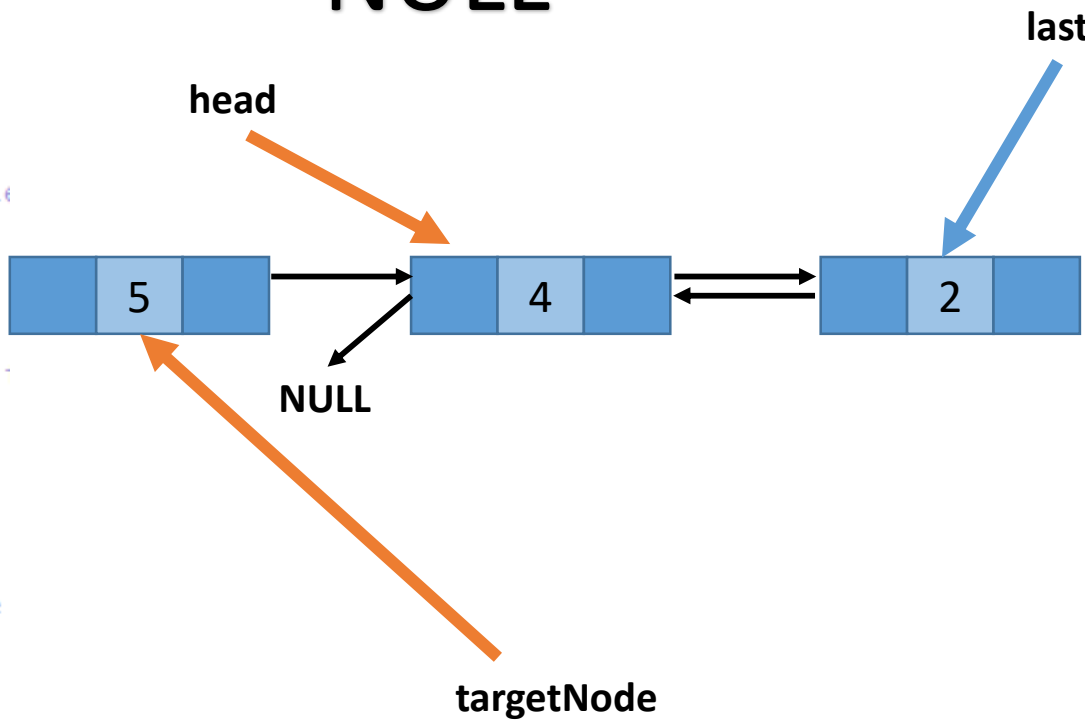
Successor Case 1: It is not NULL



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to updat
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

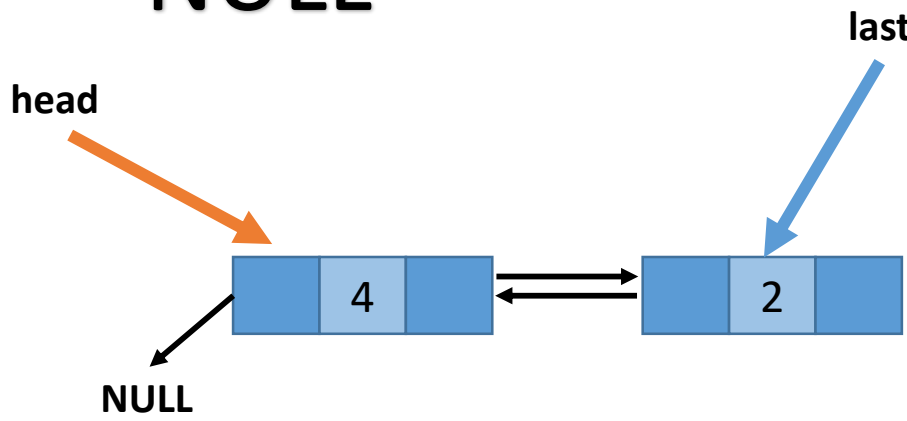
Successor Case 1: It is not NULL



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to updat
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

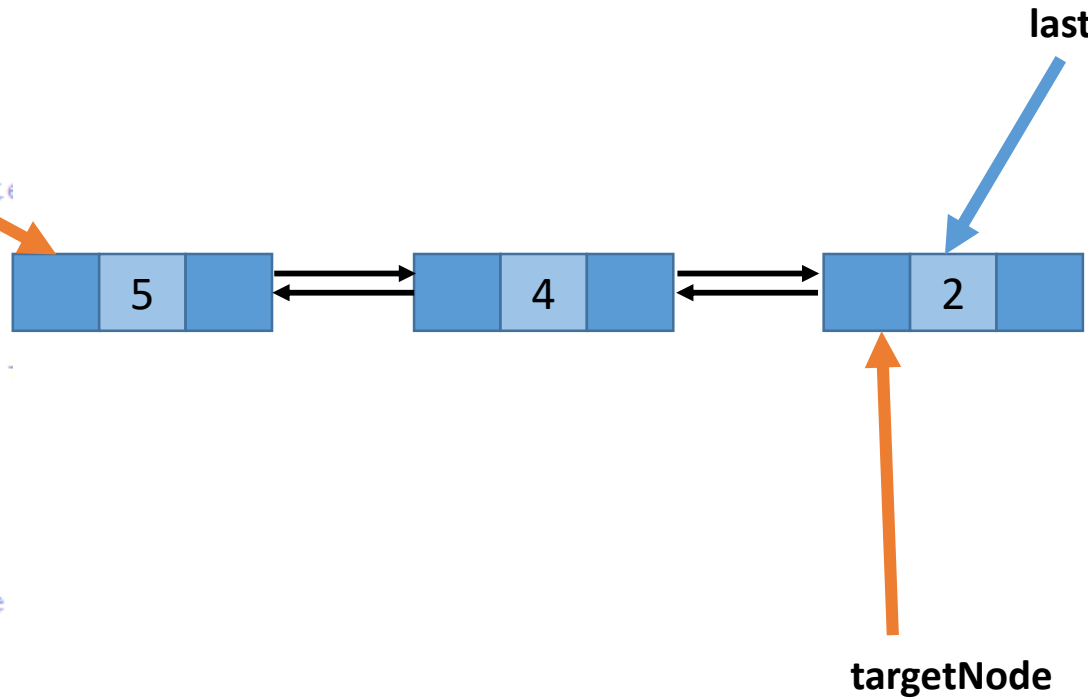
Successor Case 1: It is not NULL



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

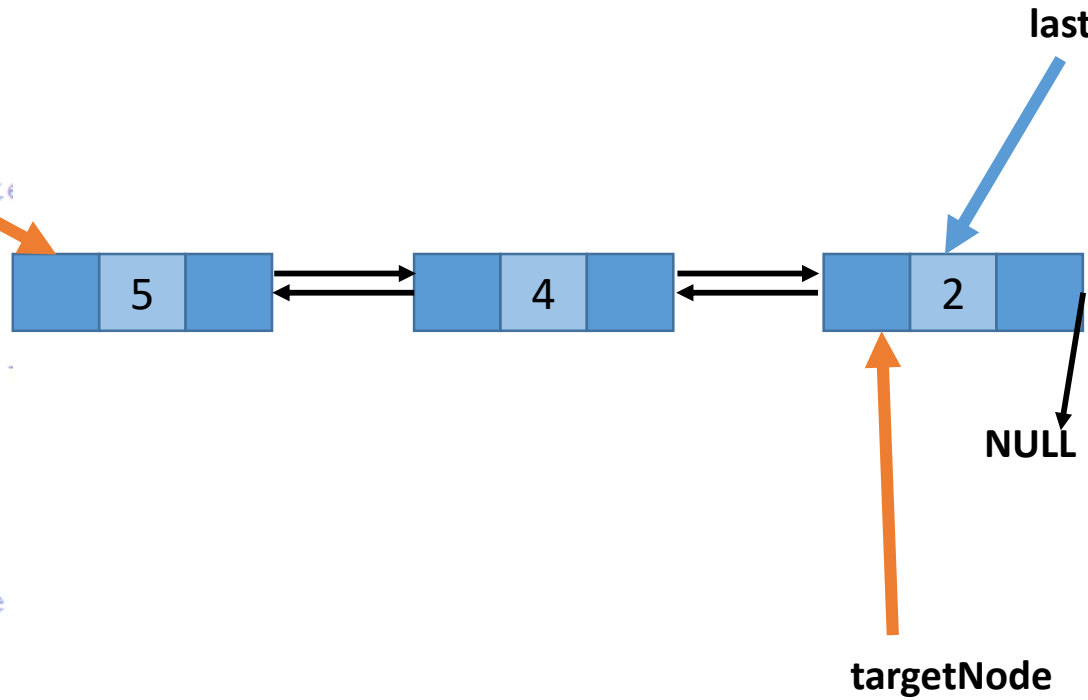
Target Case 1: It is not the first node



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

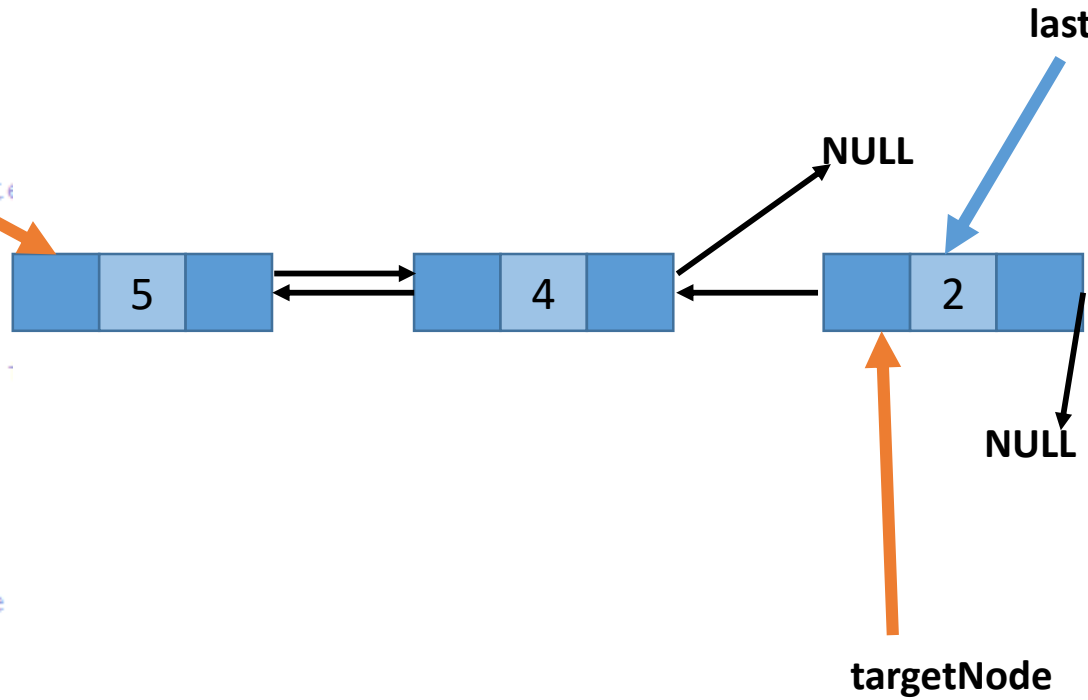
Target Case 1: It is not the first node



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

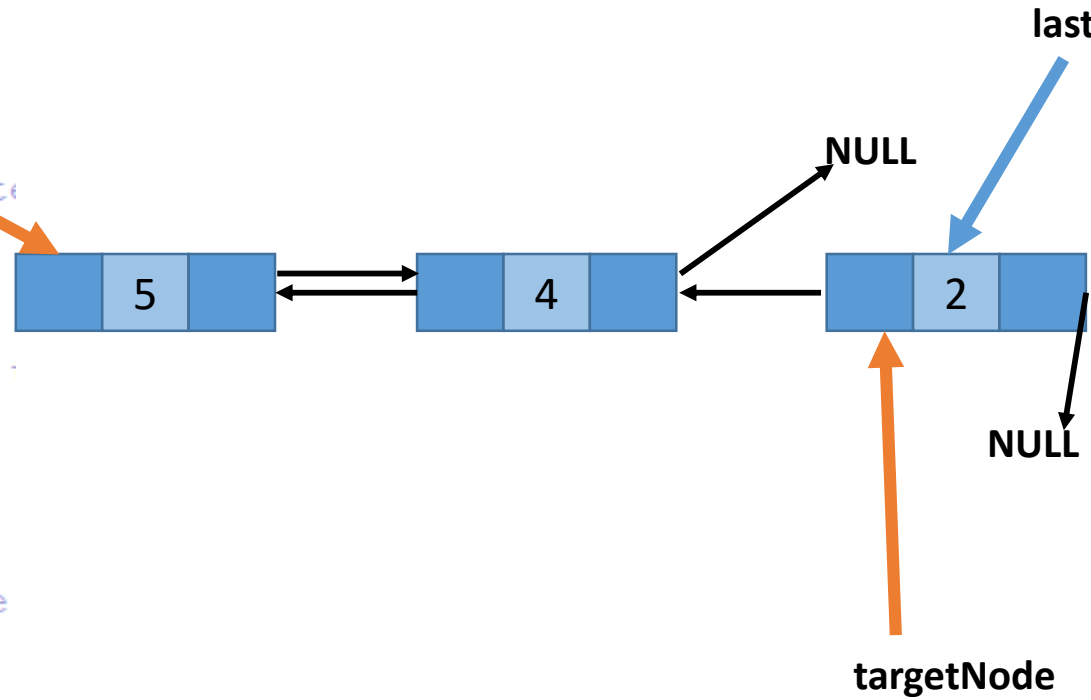
Target Case 1: It is not the first node



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

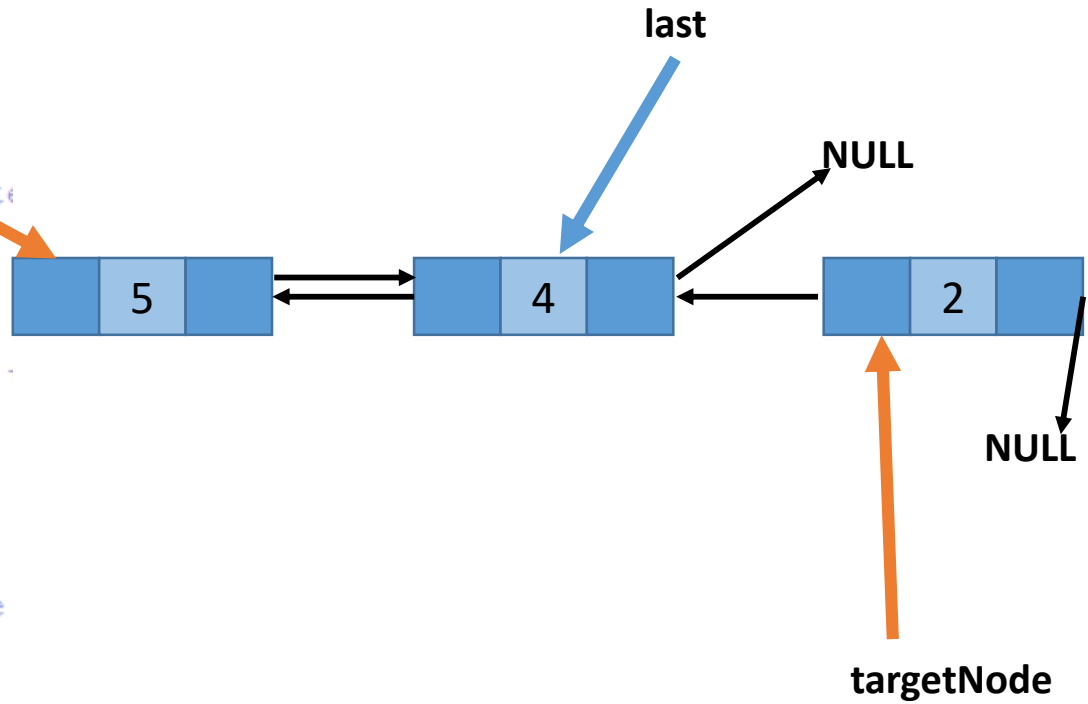
Successor Case 1: It is not NULL



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

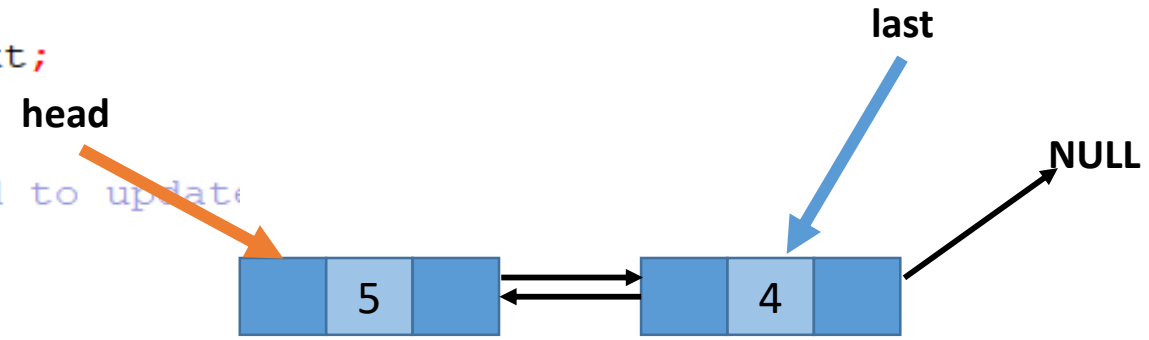
Successor Case 1: It is not NULL



Delete A Target Node

```
void delete_node(struct dnode* targetNode){
    if(targetNode == NULL){
        printf("Given node is NULL. Cannot delete.\n");
        return;
    }
    // If the node to be deleted is not the first node in
    if(targetNode->prev != NULL){
        targetNode->prev->next = targetNode->next;
    }
    else{
        // If it is the first node, then we need to upda
        head = targetNode->next;
    }

    // If the node to be deleted is not the last node in
    if(targetNode->next != NULL){
        targetNode->next->prev = targetNode->prev;
    }
    else{
        // If it is the last node, then we need to update
        last = targetNode->prev;
    }

    // Now that the node has been removed from the list,
    free(targetNode);
}
```

Successor Case 1: It is not NULL



Advantages/Disadvantages of Doubly Linked List

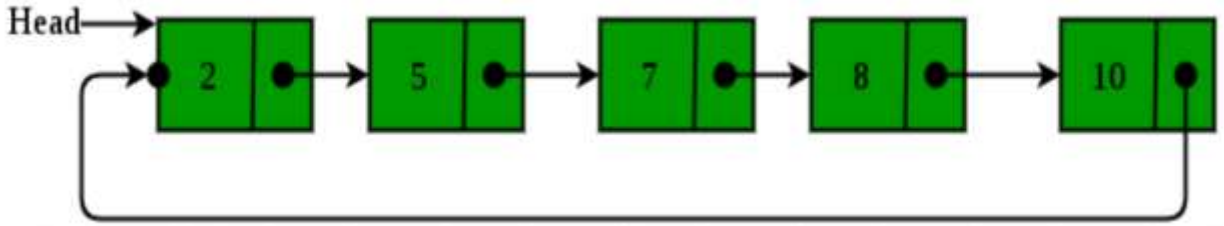
- **Advantages over singly linked list:**
  - A DLL can be traversed in both forward and backward direction.
  - We can quickly insert a new node before a given node.
  - The delete operation in DLL is more efficient if pointer to the node to be deleted is given. ○
    - In singly linked list, to delete a node, pointer to the previous node is needed. To get this previous node, sometimes the list is traversed. In DLL, we can get the previous node using ‘prev’ pointer.
- **Disadvantages over singly linked list:**
  - Every node of DLL require extra space for an previous pointer.
  - All operations require an extra pointer ‘prev’ to be maintained.

CIRCULAR LINKED LIST

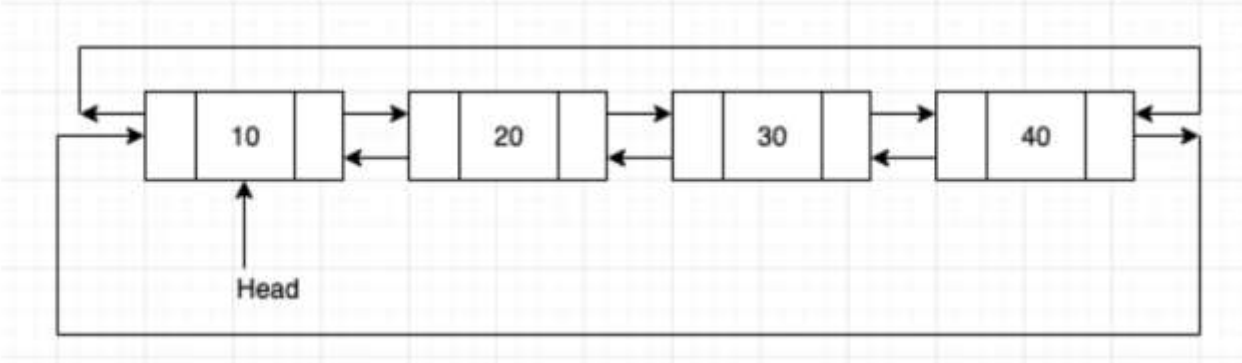
Circular Linked List

- In a circular linked list every element has a link to its next element and the last element has a link to the first element.

Singly circular linked list



Doubly circular linked list



Single Circular Linked List

- To implement a circular singly linked list, we take a global pointer that points to the **last node** *instead* of the **Head node**.
- Why ?
  - For the insertion at the beginning, the whole list has to be traversed.
  - Also, for insertion at the end, the whole list has to be traversed.
  - If we take a pointer to the last node, then in both cases there won't be any need to traverse the whole list.

Single Circular Linked List

INSERT END

```
void insert_end(int data) {
    struct node *newNode = malloc(sizeof(struct node));
    newNode->data = data;

    if(last == NULL) {
        last = newNode;
        last->next = last; // Point to itself, since it's the only node
    } else {
        newNode->next = last->next;
        last->next = newNode;
        last = newNode; // Update last pointer to new node
    }
}
```

Single Circular Linked List

INSERT END

```
void insert_end(int data) {
    struct node *newNode = malloc(sizeof(struct node));
    newNode->data = data;

    if(last == NULL) {
        last = newNode;
        last->next = last; // Point to itself, since it's the only node
    } else {
        newNode->next = last->next;
        last->next = newNode;
        last = newNode; // Update last pointer to new node
    }
}
```

Single Circular Linked List

INSERT BEGINNING

```
void insert_beginning(int data) {
    struct node *newNode = malloc(sizeof(struct node));
    newNode->data = data;

    if(last == NULL) {
        last = newNode;
        last->next = last; // Point to itself, since it's the only node
    } else {
        newNode->next = last->next;
        last->next = newNode;
    }
}
```

Single Circular Linked List

INSERT AFTER

```
void insert_beginning(int data) {
    struct node *newNode = malloc(sizeof(struct node));
    newNode->data = data;

    if(last == NULL) {
        last = newNode;
        last->next = last; // Point to itself, since it's the only node
    } else {
        newNode->next = last->next;
        last->next = newNode;
    }
}
```

Single Circular Linked List

INSERT AFTER

```
void insert_after(int data, struct node *prevNode) {
    if(prevNode == NULL) {
        printf("The given previous node cannot be NULL");
        return;
    }

    struct node *newNode = malloc(sizeof(struct node));
    newNode->data = data;

    newNode->next = prevNode->next;
    prevNode->next = newNode;

    if(prevNode == last) { // If the previous node was the last node, update 'last'
        last = newNode;
    }
}
```

SINGLE CIRCULAR LINKED LIST

```
void delete_node(struct node* targetNode) {
    // If the list is empty or the node is NULL
    if (last == NULL || targetNode == NULL) {
        printf("Cannot delete. Either list is empty or given node is NULL.\n");
        return;
    }
    // If the list only has one node
    if (last == targetNode && last->next == last) {
        last = NULL;
        free(targetNode);
        return;
    }
    struct node* current = last->next;
    struct node* previous = last;

    // Traverse the list until we find the node or we've looked at all the nodes
    while (current != last && current != targetNode) {
        previous = current;
        current = current->next;
    }

    // If we found the node
    if (current == targetNode) {
        previous->next = current->next;
        if (last == targetNode) {
            last = previous;
        }
        free(current);
        return;
    }
}
```

Summary of Linked Lists

- Advantages:
  - Linked List is dynamic data structure, that is, the Linked List grows dynamically.
  - Insertion into Linked Lists and deletion from Linked Lists are very fast (O(1) time considering that search took place).
- Disadvantages:
  - Linked List is a sequential access data structure.
  - Searching an element by pointers is very slow (O(n) time)
- A Linked List is a suitable structure when
  - A lot of insertions and deletions are required.
  - A small number of searching and retrieval are required.

Thank You